

## SOLIDWORKS API (.NET VBA C#) — Examples (49/50): Table of Wizard Hole Types and Valid Pro- Undo\_Deleted\_Note\_and\_Fire\_Undo\_Post-Not

SOLIDWORKS API Documentation (converted from CHM)

Hole Wizard Feature Types and Their Valid Properties

The properties that are valid for a specific instance of an [IWizardHoleFeatureData2](#) object depend on the type of Wizard Hole feature. To determine the type of Wizard Hole feature, use [IWizardHoleFeatureData2::Type](#). The return value is one of the valid Hole Wizard feature's types as defined in [swWzdHoleTypes\\_e](#).

The valid [IWizardHoleFeatureData2](#) properties for each Wizard Hole type are:

| Hole and Slot Types   | Valid IWizardHoleFeatureData2 Properties |
|-----------------------|------------------------------------------|
| swSimple              | <a href="#">Diameter</a>                 |
|                       | <a href="#">Depth</a>                    |
|                       | <a href="#">Length</a> (slot only)       |
|                       |                                          |
| swTapered             | <a href="#">MinorDiameter</a>            |
|                       | <a href="#">Depth</a>                    |
|                       | <a href="#">MajorDiameter</a>            |
|                       |                                          |
| swCounterBored        | <a href="#">Diameter</a>                 |
|                       | <a href="#">Depth</a>                    |
|                       | <a href="#">CounterBoreDiameter</a>      |
|                       | <a href="#">CounterBoreDepth</a>         |
|                       | <a href="#">Length</a> (slot only)       |
| swCounterSunk         | <a href="#">Diameter</a>                 |
|                       | <a href="#">Depth</a>                    |
|                       | <a href="#">CounterSinkAngle</a>         |
|                       | <a href="#">CounterSinkDiameter</a>      |
|                       | <a href="#">Length</a> (slot only)       |
| swCounterDrilled      | <a href="#">Diameter</a>                 |
|                       | <a href="#">Depth</a>                    |
|                       | <a href="#">CounterDrillDiameter</a>     |
|                       | <a href="#">CounterDrillDepth</a>        |
|                       | <a href="#">CounterDrillAngle</a>        |
| swSimpleDrilled       | <a href="#">Diameter</a>                 |
|                       | <a href="#">Depth</a>                    |
|                       | <a href="#">DrillAngle</a>               |
|                       |                                          |
| swTaperedDrilled      | <a href="#">MinorDiameter</a>            |
|                       | <a href="#">Depth</a>                    |
|                       | <a href="#">MajorDiameter</a>            |
|                       | <a href="#">DrillAngle</a>               |
|                       |                                          |
| swCounterBoredDrilled | <a href="#">Diameter</a>                 |
|                       | <a href="#">Depth</a>                    |
|                       | <a href="#">CounterBoreDiameter</a>      |
|                       | <a href="#">CounterBoreDepth</a>         |
|                       | <a href="#">DrillAngle</a>               |
| swCounterSunkDrilled  | <a href="#">Diameter</a>                 |
|                       | <a href="#">Depth</a>                    |
|                       | <a href="#">CounterSinkDiameter</a>      |
|                       | <a href="#">CounterSinkAngle</a>         |

|                                        |                                         |
|----------------------------------------|-----------------------------------------|
|                                        | <a href="#">DrillAngle</a>              |
| swCounterDrilledDrilled                | <a href="#">Diameter</a>                |
|                                        | <a href="#">Depth</a>                   |
|                                        | <a href="#">CounterDrillDiameter</a>    |
|                                        | <a href="#">CounterDrillDepth</a>       |
|                                        | <a href="#">DrillAngle</a>              |
|                                        | <a href="#">CounterDrillAngle</a>       |
| swCounterBoreBlind                     | <a href="#">HoleDiameter</a>            |
|                                        | <a href="#">HoleDepth</a>               |
|                                        | <a href="#">CounterBoreDiameter</a>     |
|                                        | <a href="#">CounterBoreDepth</a>        |
|                                        | <a href="#">DrillAngle</a>              |
|                                        | <a href="#">Length</a> (slot only)      |
| swCounterBoreBlindCounterSinkMiddle    | <a href="#">HoleDiameter</a>            |
|                                        | <a href="#">HoleDepth</a>               |
|                                        | <a href="#">CounterBoreDiameter</a>     |
|                                        | <a href="#">CounterBoreDepth</a>        |
|                                        | <a href="#">MidCounterSinkDiameter</a>  |
|                                        | <a href="#">MidCounterSinkAngle</a>     |
|                                        | <a href="#">DrillAngle</a>              |
|                                        | <a href="#">Length</a> (slot only)      |
| swCounterBoreBlindCounterSinkTop       | <a href="#">HoleDiameter</a>            |
|                                        | <a href="#">HoleDepth</a>               |
|                                        | <a href="#">CounterBoreDiameter</a>     |
|                                        | <a href="#">CounterBoreDepth</a>        |
|                                        | <a href="#">NearCounterSinkDiameter</a> |
|                                        | <a href="#">NearCounterSinkAngle</a>    |
|                                        | <a href="#">DrillAngle</a>              |
|                                        | <a href="#">Length</a> (slot only)      |
| swCounterBoreBlindCounterSinkTopmiddle | <a href="#">HoleDiameter</a>            |
|                                        | <a href="#">HoleDepth</a>               |
|                                        | <a href="#">CounterBoreDiameter</a>     |
|                                        | <a href="#">CounterBoreDepth</a>        |
|                                        | <a href="#">NearCounterSinkDiameter</a> |
|                                        | <a href="#">NearCounterSinkAngle</a>    |
|                                        | <a href="#">MidCounterSinkDiameter</a>  |
|                                        | <a href="#">MidCounterSinkAngle</a>     |
|                                        | <a href="#">DrillAngle</a>              |
|                                        | <a href="#">Length</a> (slot only)      |
| swCounterBoreThru                      | <a href="#">ThruHoleDiameter</a>        |
|                                        | <a href="#">ThruHoleDepth</a>           |
|                                        | <a href="#">CounterBoreDiameter</a>     |
|                                        | <a href="#">CounterBoreDepth</a>        |
|                                        | <a href="#">Length</a> (slot only)      |

|                                          |                                         |
|------------------------------------------|-----------------------------------------|
| swCounterBoreThruCounterSinkBottom       | <a href="#">ThruHoleDiameter</a>        |
|                                          | <a href="#">ThruHoleDepth</a>           |
|                                          | <a href="#">CounterBoreDiameter</a>     |
|                                          | <a href="#">CounterBoreDepth</a>        |
|                                          | <a href="#">FarCounterSinkDiameter</a>  |
|                                          | <a href="#">FarCounterSinkAngle</a>     |
|                                          | <a href="#">Length</a> (slot only)      |
| swCounterBoreThruCounterSinkMiddle       | <a href="#">ThruHoleDiameter</a>        |
|                                          | <a href="#">ThruHoleDepth</a>           |
|                                          | <a href="#">CounterBoreDiameter</a>     |
|                                          | <a href="#">CounterBoreDepth</a>        |
|                                          | <a href="#">MidCounterSinkDiameter</a>  |
|                                          | <a href="#">MidCounterSinkAngle</a>     |
|                                          | <a href="#">Length</a> (slot only)      |
| swCounterBoreThruCounterSinkMiddleBottom | <a href="#">ThruHoleDiameter</a>        |
|                                          | <a href="#">ThruHoleDepth</a>           |
|                                          | <a href="#">CounterBoreDiameter</a>     |
|                                          | <a href="#">CounterBoreDepth</a>        |
|                                          | <a href="#">MidCounterSinkDiameter</a>  |
|                                          | <a href="#">MidCounterSinkAngle</a>     |
|                                          | <a href="#">FarCounterSinkDiameter</a>  |
| swCounterBoreThruCounterSinkTop          | <a href="#">ThruHoleDiameter</a>        |
|                                          | <a href="#">ThruHoleDepth</a>           |
|                                          | <a href="#">CounterBoreDiameter</a>     |
|                                          | <a href="#">CounterBoreDepth</a>        |
|                                          | <a href="#">NearCounterSinkDiameter</a> |
|                                          | <a href="#">NearCounterSinkAngle</a>    |
|                                          | <a href="#">Length</a> (slot only)      |
| swCounterBoreThruCounterSinkTopBottom    | <a href="#">ThruHoleDiameter</a>        |
|                                          | <a href="#">ThruHoleDepth</a>           |
|                                          | <a href="#">CounterBoreDiameter</a>     |
|                                          | <a href="#">CounterBoreDepth</a>        |
|                                          | <a href="#">NearCounterSinkDiameter</a> |
|                                          | <a href="#">NearCounterSinkAngle</a>    |
|                                          | <a href="#">FarCounterSinkDiameter</a>  |
| swCounterBoreThruCounterSinkTopMiddle    | <a href="#">ThruHoleDiameter</a>        |
|                                          | <a href="#">ThruHoleDepth</a>           |
|                                          | <a href="#">CounterBoreDiameter</a>     |
|                                          | <a href="#">CounterBoreDepth</a>        |
|                                          |                                         |
|                                          |                                         |
|                                          |                                         |

|                                             |                                         |
|---------------------------------------------|-----------------------------------------|
|                                             | <a href="#">NearCounterSinkDiameter</a> |
|                                             | <a href="#">NearCounterSinkAngle</a>    |
|                                             | <a href="#">MidCounterSinkDiameter</a>  |
|                                             | <a href="#">MidCounterSinkAngle</a>     |
|                                             | <a href="#">Length</a> (slot only)      |
| swCounterBoreThruCounterSinkTopMiddleBottom | <a href="#">ThruHoleDiameter</a>        |
|                                             | <a href="#">ThruHoleDepth</a>           |
|                                             | <a href="#">CounterBoreDiameter</a>     |
|                                             | <a href="#">CounterBoreDepth</a>        |
|                                             | <a href="#">NearCounterSinkDiameter</a> |
|                                             | <a href="#">NearCounterSinkAngle</a>    |
|                                             | <a href="#">MidCounterSinkDiameter</a>  |
|                                             | <a href="#">MidCounterSinkAngle</a>     |
|                                             | <a href="#">FarCounterSinkDiameter</a>  |
|                                             | <a href="#">FarCounterSinkAngle</a>     |
|                                             | <a href="#">Length</a> (slot only)      |
| swHoleBlind                                 | <a href="#">HoleDiameter</a>            |
|                                             | <a href="#">HoleDepth</a>               |
|                                             | <a href="#">DrillAngle</a>              |
|                                             | <a href="#">Length</a> (slot only)      |
| swHoleBlindCounterSinkTop                   | <a href="#">HoleDiameter</a>            |
|                                             | <a href="#">HoleDepth</a>               |
|                                             | <a href="#">NearCounterSinkDiameter</a> |
|                                             | <a href="#">NearCounterSinkAngle</a>    |
|                                             | <a href="#">DrillAngle</a>              |
|                                             | <a href="#">Length</a> (slot only)      |
| swCounterSinkBlind                          | <a href="#">HoleDiameter</a>            |
|                                             | <a href="#">HoleDepth</a>               |
|                                             | <a href="#">CounterSinkDiameter</a>     |
|                                             | <a href="#">CounterSinkAngle</a>        |
|                                             | <a href="#">DrillAngle</a>              |
|                                             | <a href="#">HeadClearance</a>           |
|                                             | <a href="#">Length</a> (slot only)      |
| swHoleThru                                  | <a href="#">ThruHoleDiameter</a>        |
|                                             | <a href="#">ThruHoleDepth</a>           |
|                                             | <a href="#">Length</a> (slot only)      |
| swHoleThruCounterSinkBottom                 | <a href="#">ThruHoleDiameter</a>        |
|                                             | <a href="#">ThruHoleDepth</a>           |
|                                             | <a href="#">FarCounterSinkDiameter</a>  |
|                                             | <a href="#">FarCounterSinkAngle</a>     |
|                                             | <a href="#">Length</a> (slot only)      |
| swHoleThruCounterSinkTop                    | <a href="#">ThruHoleDiameter</a>        |

|                                    |                                         |
|------------------------------------|-----------------------------------------|
|                                    | <a href="#">ThruHoleDepth</a>           |
|                                    | <a href="#">NearCounterSinkDiameter</a> |
|                                    | <a href="#">NearCounterSinkAngle</a>    |
|                                    | <a href="#">Length</a> (slot only)      |
| swHoleThruCounterSinkTopBottom     | <a href="#">ThruHoleDiameter</a>        |
|                                    | <a href="#">ThruHoleDepth</a>           |
|                                    | <a href="#">NearCounterSinkDiameter</a> |
|                                    | <a href="#">NearCounterSinkAngle</a>    |
|                                    | <a href="#">FarCounterSinkDiameter</a>  |
|                                    | <a href="#">FarCounterSinkAngle</a>     |
|                                    | <a href="#">Length</a> (slot only)      |
| swCounterSinkThru                  | <a href="#">ThruHoleDiameter</a>        |
|                                    | <a href="#">ThruHoleDepth</a>           |
|                                    | <a href="#">CounterSinkDiameter</a>     |
|                                    | <a href="#">CounterSinkAngle</a>        |
|                                    | <a href="#">HeadClearance</a>           |
|                                    | <a href="#">Length</a> (slot only)      |
| swCounterSinkThruCounterSinkBottom | <a href="#">ThruHoleDiameter</a>        |
|                                    | <a href="#">ThruHoleDepth</a>           |
|                                    | <a href="#">CounterSinkDiameter</a>     |
|                                    | <a href="#">CounterSinkAngle</a>        |
|                                    | <a href="#">FarCounterSinkDiameter</a>  |
|                                    | <a href="#">FarCounterSinkAngle</a>     |
|                                    | <a href="#">HeadClearance</a>           |
|                                    | <a href="#">Length</a> (slot only)      |
| swTapBlind                         | <a href="#">TapDrillDiameter</a>        |
|                                    | <a href="#">TapDrillDepth</a>           |
|                                    | <a href="#">ThreadDiameter</a>          |
|                                    | <a href="#">ThreadDepth</a>             |
|                                    | <a href="#">DrillAngle</a>              |
| swTapBlindCounterSinkTop           | <a href="#">TapDrillDiameter</a>        |
|                                    | <a href="#">TapDrillDepth</a>           |
|                                    | <a href="#">ThreadDiameter</a>          |
|                                    | <a href="#">ThreadDepth</a>             |
|                                    | <a href="#">NearCounterSinkDiameter</a> |
|                                    | <a href="#">NearCounterSinkAngle</a>    |
|                                    | <a href="#">DrillAngle</a>              |
| swTapThru                          | <a href="#">ThruTapDrillDiameter</a>    |
|                                    | <a href="#">ThruTapDrillDepth</a>       |
|                                    | <a href="#">ThreadDiameter</a>          |
|                                    | <a href="#">ThreadDepth</a>             |
| swTapThruCounterSinkBottom         | <a href="#">ThruTapDrillDiameter</a>    |

|                                |                                         |
|--------------------------------|-----------------------------------------|
|                                | <a href="#">ThruTapDrillDepth</a>       |
|                                | <a href="#">ThreadDiameter</a>          |
|                                | <a href="#">ThreadDepth</a>             |
|                                | <a href="#">FarCounterSinkDiameter</a>  |
|                                | <a href="#">FarCounterSinkAngle</a>     |
|                                |                                         |
| swTapThruCounterSinkTop        | <a href="#">ThruTapDrillDiameter</a>    |
|                                | <a href="#">ThruTapDrillDepth</a>       |
|                                | <a href="#">ThreadDiameter</a>          |
|                                | <a href="#">ThreadDepth</a>             |
|                                | <a href="#">NearCounterSinkDiameter</a> |
|                                | <a href="#">NearCounterSinkAngle</a>    |
|                                |                                         |
| swTapThruCounterSinkTopBottom  | <a href="#">ThruTapDrillDiameter</a>    |
|                                | <a href="#">ThruTapDrillDepth</a>       |
|                                | <a href="#">ThreadDiameter</a>          |
|                                | <a href="#">ThreadDepth</a>             |
|                                | <a href="#">NearCounterSinkDiameter</a> |
|                                | <a href="#">NearCounterSinkAngle</a>    |
|                                | <a href="#">FarCounterSinkDiameter</a>  |
|                                | <a href="#">FarCounterSinkAngle</a>     |
|                                |                                         |
| swPipeTapBlind                 | <a href="#">TapDrillDiameter</a>        |
|                                | <a href="#">TapDrillDepth</a>           |
|                                | <a href="#">ThreadDiameter</a>          |
|                                | <a href="#">ThreadAngle</a>             |
|                                | <a href="#">ThreadDepth</a>             |
|                                | <a href="#">DrillAngle</a>              |
|                                |                                         |
| swPipeTapBlindCounterSinkTop   | <a href="#">TapDrillDiameter</a>        |
|                                | <a href="#">TapDrillDepth</a>           |
|                                | <a href="#">ThreadDiameter</a>          |
|                                | <a href="#">ThreadAngle</a>             |
|                                | <a href="#">ThreadDepth</a>             |
|                                | <a href="#">CounterSinkDiameter</a>     |
|                                | <a href="#">CounterSinkAngle</a>        |
|                                | <a href="#">DrillAngle</a>              |
|                                |                                         |
| swPipeTapThru                  | <a href="#">ThruTapDrillDiameter</a>    |
|                                | <a href="#">ThruTapDrillDepth</a>       |
|                                | <a href="#">ThreadDiameter</a>          |
|                                | <a href="#">ThreadAngle</a>             |
|                                | <a href="#">ThreadDepth</a>             |
|                                |                                         |
| swPipeTapThruCounterSinkBottom | <a href="#">ThruTapDrillDiameter</a>    |
|                                | <a href="#">ThruTapDrillDepth</a>       |
|                                | <a href="#">ThreadDiameter</a>          |
|                                | <a href="#">ThreadAngle</a>             |
|                                | <a href="#">ThreadDepth</a>             |

|                                                        |                                         |
|--------------------------------------------------------|-----------------------------------------|
|                                                        | <a href="#">FarCounterSinkDiameter</a>  |
|                                                        | <a href="#">FarCounterSinkAngle</a>     |
|                                                        |                                         |
| swPipeTapThruCounterSinkTop                            | <a href="#">ThruTapDrillDiameter</a>    |
|                                                        | <a href="#">ThruTapDrillDepth</a>       |
|                                                        | <a href="#">ThreadDiameter</a>          |
|                                                        | <a href="#">ThreadAngle</a>             |
|                                                        | <a href="#">ThreadDepth</a>             |
|                                                        | <a href="#">NearCounterSinkDiameter</a> |
|                                                        | <a href="#">NearCounterSinkAngle</a>    |
|                                                        |                                         |
| swPipeTapThruCounterSinkTopBottom                      | <a href="#">ThruTapDrillDiameter</a>    |
|                                                        | <a href="#">ThruTapDrillDepth</a>       |
|                                                        | <a href="#">ThreadDiameter</a>          |
|                                                        | <a href="#">ThreadAngle</a>             |
|                                                        | <a href="#">ThreadDepth</a>             |
|                                                        | <a href="#">NearCounterSinkDiameter</a> |
|                                                        | <a href="#">NearCounterSinkAngle</a>    |
|                                                        | <a href="#">FarCounterSinkDiameter</a>  |
|                                                        | <a href="#">FarCounterSinkAngle</a>     |
|                                                        |                                         |
| swCounterSinkBlindWithoutHeadClearance                 | <a href="#">HoleDiameter</a>            |
|                                                        | <a href="#">HoleDepth</a>               |
|                                                        | <a href="#">CounterSinkDiameter</a>     |
|                                                        | <a href="#">CounterSinkAngle</a>        |
|                                                        | <a href="#">DrillAngle</a>              |
|                                                        | <a href="#">Length</a> (slot only)      |
|                                                        |                                         |
| swCounterSinkThruWithoutHeadClearance                  | <a href="#">ThruHoleDiameter</a>        |
|                                                        | <a href="#">ThruHoleDepth</a>           |
|                                                        | <a href="#">CounterSinkDiameter</a>     |
|                                                        | <a href="#">CounterSinkAngle</a>        |
|                                                        | <a href="#">Length</a> (slot only)      |
|                                                        |                                         |
| swCounterSinkThruCounterSinkBottomWithoutHeadClearance | <a href="#">ThruHoleDiameter</a>        |
|                                                        | <a href="#">ThruHoleDepth</a>           |
|                                                        | <a href="#">CounterSinkDiameter</a>     |
|                                                        | <a href="#">CounterSinkAngle</a>        |
|                                                        | <a href="#">FarCounterSinkDiameter</a>  |
|                                                        | <a href="#">FarCounterSinkAngle</a>     |
|                                                        | <a href="#">HeadClearance</a>           |
|                                                        | <a href="#">Length</a> (slot only)      |
|                                                        |                                         |
| swTapBlindCosmeticThread                               | <a href="#">TapDrillDiameter</a>        |
|                                                        | <a href="#">TapDrillDepth</a>           |
|                                                        | <a href="#">ThreadDiameter</a>          |
|                                                        | <a href="#">ThreadDepth</a>             |
|                                                        | <a href="#">DrillAngle</a>              |
|                                                        |                                         |

|                                             |                                         |
|---------------------------------------------|-----------------------------------------|
| swTapBlindCosmeticThreadCounterSinkTop      | <a href="#">TapDrillDiameter</a>        |
|                                             | <a href="#">TapDrillDepth</a>           |
|                                             | <a href="#">ThreadDiameter</a>          |
|                                             | <a href="#">ThreadDepth</a>             |
|                                             | <a href="#">DrillAngle</a>              |
|                                             | <a href="#">NearCounterSinkDiameter</a> |
|                                             | <a href="#">NearCounterSinkAngle</a>    |
| swTapThruCosmeticThread                     | <a href="#">ThruTapDrillDiameter</a>    |
|                                             | <a href="#">ThruTapDrillDepth</a>       |
|                                             | <a href="#">ThreadDiameter</a>          |
|                                             | <a href="#">ThreadDepth</a>             |
|                                             | <a href="#">DrillAngle</a>              |
| swTapThruCosmeticThreadCounterSinkTop       | <a href="#">ThruTapDrillDiameter</a>    |
|                                             | <a href="#">ThruTapDrillDepth</a>       |
|                                             | <a href="#">ThreadDiameter</a>          |
|                                             | <a href="#">ThreadDepth</a>             |
|                                             | <a href="#">DrillAngle</a>              |
|                                             | <a href="#">NearCounterSinkDiameter</a> |
|                                             | <a href="#">NearCounterSinkAngle</a>    |
| swTapThruCosmeticThreadCounterSinkBottom    | <a href="#">ThruTapDrillDiameter</a>    |
|                                             | <a href="#">ThruTapDrillDepth</a>       |
|                                             | <a href="#">ThreadDiameter</a>          |
|                                             | <a href="#">ThreadDepth</a>             |
|                                             | <a href="#">DrillAngle</a>              |
|                                             | <a href="#">FarCounterSinkDiameter</a>  |
|                                             | <a href="#">FarCounterSinkAngle</a>     |
| swTapThruCosmeticThreadCounterSinkTopBottom | <a href="#">ThruTapDrillDiameter</a>    |
|                                             | <a href="#">ThruTapDrillDepth</a>       |
|                                             | <a href="#">ThreadDiameter</a>          |
|                                             | <a href="#">ThreadDepth</a>             |
|                                             | <a href="#">DrillAngle</a>              |
|                                             | <a href="#">FarCounterSinkDiameter</a>  |
|                                             | <a href="#">FarCounterSinkAngle</a>     |
|                                             | <a href="#">NearCounterSinkDiameter</a> |
|                                             | <a href="#">NearCounterSinkAngle</a>    |
| swTapThruThreadThru                         | <a href="#">ThruTapDrillDiameter</a>    |
|                                             | <a href="#">ThruTapDrillDepth</a>       |
|                                             | <a href="#">ThruHoleDepth</a>           |
|                                             | <a href="#">ThruHoleDiameter</a>        |
|                                             | <a href="#">ThreadDiameter</a>          |
|                                             | <a href="#">ThreadDepth</a>             |
|                                             | <a href="#">ThreadEndCondition</a>      |
| swTapThruThreadThruCounterSinkTop           | <a href="#">ThruTapDrillDiameter</a>    |
|                                             | <a href="#">ThruTapDrillDepth</a>       |

|                                         |                                         |
|-----------------------------------------|-----------------------------------------|
|                                         | <a href="#">ThruHoleDepth</a>           |
|                                         | <a href="#">ThruHoleDiameter</a>        |
|                                         | <a href="#">ThreadDiameter</a>          |
|                                         | <a href="#">ThreadDepth</a>             |
|                                         | <a href="#">ThreadEndCondition</a>      |
|                                         | <a href="#">NearCounterSinkDiameter</a> |
|                                         | <a href="#">NearCounterSinkAngle</a>    |
|                                         |                                         |
| swTapThruThreadThruCounterSinkBottom    | <a href="#">ThruTapDrillDiameter</a>    |
|                                         | <a href="#">ThruTapDrillDepth</a>       |
|                                         | <a href="#">ThruHoleDepth</a>           |
|                                         | <a href="#">ThruHoleDiameter</a>        |
|                                         | <a href="#">ThreadDiameter</a>          |
|                                         | <a href="#">ThreadDepth</a>             |
|                                         | <a href="#">ThreadEndCondition</a>      |
|                                         | <a href="#">FarCounterSinkDiameter</a>  |
|                                         | <a href="#">FarCounterSinkAngle</a>     |
|                                         |                                         |
| swTapThruThreadThruCountersinkTopBottom | <a href="#">ThruTapDrillDiameter</a>    |
|                                         | <a href="#">ThruTapDrillDepth</a>       |
|                                         | <a href="#">ThruHoleDepth</a>           |
|                                         | <a href="#">ThruHoleDiameter</a>        |
|                                         | <a href="#">ThreadDiameter</a>          |
|                                         | <a href="#">ThreadDepth</a>             |
|                                         | <a href="#">ThreadEndCondition</a>      |
|                                         | <a href="#">FarCounterSinkDiameter</a>  |
|                                         | <a href="#">FarCounterSinkAngle</a>     |
|                                         | <a href="#">NearCounterSinkDiameter</a> |
|                                         | <a href="#">NearCounterSinkAngle</a>    |
|                                         |                                         |
| swTapBlindRemoveThread                  | <a href="#">TapDrillDiameter</a>        |
|                                         | <a href="#">TapDrillDepth</a>           |
|                                         |                                         |

## Temporarily Fix and Group Components Example (C#)

This example shows how to temporarily fix and group components.

```
//-----
// Preconditions: Verify that the specified assembly exists.
//
// Postconditions:
// 1. Opens the specified assembly, changes the view orientation to
//    dimetric, and zooms and fits the assembly in the graphics area.
// 2. Selects the rocker-1@valve_cam component.
//    a. Temporarily changes the rocker-1@valve_cam component from
//       floating to fixed, which is indicated by (f)* appearing before
//       rocker-1@valve_cam in the FeatureManager design tree and
//       the pink component in the graphics area.
//    b. Press F5 after examining the FeatureManager design tree and
//       graphics area.
//    c. Changes the rocker-1@valve_cam component back to floating.
// 3. Selects the rocker-1@valve_cam and camshaft-1@valve_cam
//    components.
//    a. Temporarily groups the rocker-1@valve_cam and
//       camshaft-1@valve_cam components, which is indicated by
//       the pink components in the graphics area.
//    b. Press F5 after examining the graphics area.
//    c. Ungroups the rocker-1@valve_cam and camshaft-1@valve_cam
//       components.
// 4. Examine both the FeatureManager design tree and graphics area to
//    verify steps 2.c and 3.c.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            AssemblyDoc swAssembly = default(AssemblyDoc);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\motionstudies\\valve_cam.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref e
            rrors, ref warnings);
            swAssembly = (AssemblyDoc)swModel;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swModel.ShowNamedView2("Dimetric", 9);
            swModel.ViewZoomToFit2();

            swModel.ClearSelection2(true);

            //Temporarily fix a component
            status = swModelDocExt.SelectByID2("rocker-1@valve_cam", "COMPONENT", 0, 0, 0, true, 0, null, 0);

            swAssembly.TemporaryFixGroup();

            System.Diagnostics.Debugger.Break();
            //Examine rocker-1@valve_cam in the FeatureManager design tree and graphics area
            //(f)* and the pink component indicate that the component is fixed
            //Press F5

            //Changes the component back to floating
            swAssembly.TemporaryFixGroupExit();

            swModel.ClearSelection2(true);

            //Temporarily group components
            status = swModelDocExt.SelectByID2("camshaft-1@valve_cam", "COMPONENT", 0, 0, 0, true, 2, null, 0);
            status = swModelDocExt.SelectByID2("rocker-1@valve_cam", "COMPONENT", 0, 0, 0, true, 2, null, 0);
            swAssembly.TemporaryFixGroup();

            System.Diagnostics.Debugger.Break();
            //Examine the graphics area
            //Pink components indicate that the components are grouped
            //Press F5

            //Ungroup components
            swAssembly.TemporaryFixGroupExit();
        }
    }
    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
```

## Temporarily Fix and Group Components Example (VBA)

This example shows how to temporarily fix and group components.

```
'-----
' Preconditions: Verify that the specified assembly exists.
'
' Postconditions:
' 1. Opens the specified assembly, changes the view orientation to
'    dimetric, and zooms and fits the assembly in the graphics area.
' 2. Selects the rocker-1@valve_cam component.
'    a. Temporarily changes the rocker-1@valve_cam component from
'       floating to fixed, which is indicated by (f)* appearing before
'       rocker-1@valve_cam in the FeatureManager design tree and
'       the pink component in the graphics area.
'    b. Press F5 after examining the FeatureManager design tree and
'       graphics area.
'    c. Changes the rocker-1@valve_cam component back to floating.
' 3. Selects the rocker-1@valve_cam and camshaft-1@valve_cam
'    components.
'    a. Temporarily groups the rocker-1@valve_cam and
'       camshaft-1@valve_cam components, which is indicated by
'       the pink components in the graphics area.
'    b. Press F5 after examining the graphics area.
'    c. Ungroups the rocker-1@valve_cam and camshaft-1@valve_cam
'       components.
' 4. Examine both the FeatureManager design tree and graphics area to
'    verify steps 2.c and 3.c.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
```

## Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssembly As SldWorks.AssemblyDoc
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\motionstudies\valve_cam.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swAssembly = swModel
    Set swModelDocExt = swModel.Extension

    swModel.ShowNamedView2 "Dimetric", 9
    swModel.ViewZoomtofit2

    swModel.ClearSelection2 True

    'Temporarily fix a component
    status = swModelDocExt.SelectByID2("rocker-1@valve_cam", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)

    swAssembly.TemporaryFixGroup

    Stop
    'Examine rocker-1@valve_cam in the FeatureManager design tree and graphics area
    '(f)* and the pink component indicate that the component is fixed
    'Press F5

    'Changes the component back to floating
    swAssembly.TemporaryFixGroupExit

    swModel.ClearSelection2 True

    'Temporarily group components
    status = swModelDocExt.SelectByID2("camshaft-1@valve_cam", "COMPONENT", 0, 0, 0, True, 2, Nothing, 0)
    status = swModelDocExt.SelectByID2("rocker-1@valve_cam", "COMPONENT", 0, 0, 0, True, 2, Nothing, 0)
    swAssembly.TemporaryFixGroup

    Stop
    'Examine the graphics area
    'Pink components indicate that the components are grouped
    'Press F5

    'Ungroup components
    swAssembly.TemporaryFixGroupExit

End Sub
```

## Temporarily Fix and Group Components Example (VB.NET)

This example shows how to temporarily fix and group components.

Preconditions: Verify that the specified assembly exists.

Postconditions:

1. Opens the specified assembly, changes the view orientation to dimetric, and zooms and fits the assembly in the graphics area.
2. Selects the **rocker-1@valve\_cam** component.
  - a. Temporarily changes the **rocker-1@valve\_cam** component from floating to fixed, which is indicated by (f)\* appearing before **rocker-1@valve\_cam** in the FeatureManager design tree and the pink component in the graphics area.
  - b. Press F5 after examining the FeatureManager design tree and graphics area.
  - c. Changes the **rocker-1@valve\_cam** component back to floating.
3. Selects the **rocker-1@valve\_cam** and **camshaft-1@valve\_cam** components.
  - a. Temporarily groups the **rocker-1@valve\_cam** and **camshaft-1@valve\_cam** components, which is indicated by the pink components in the graphics area.
  - b. Press F5 after examining the graphics area.
  - c. Ungroups the **rocker-1@valve\_cam** and **camshaft-1@valve\_cam** components.
4. Examine both the FeatureManager design tree and graphics area to verify steps 2.c and 3.c.

NOTE: Because the assembly is used elsewhere, do not save changes.

Imports SolidWorks.Interop.sldworks

Imports SolidWorks.Interop.swconst

Imports System.Runtime.InteropServices

Imports System

Partial Class SolidWorksMacro

Public Sub main()

Dim swModel As ModelDoc2

Dim swAssembly As AssemblyDoc

Dim swModelDocExt As ModelDocExtension

Dim fileName As String

Dim status As Boolean

Dim errors As Integer

Dim warnings As Integer

fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\motionstudies\valve\_cam.sldasm"

swModel = swApp.OpenDoc6(fileName, swDocumentTypes\_e.swDocASSEMBLY, swOpenDocOptions\_e.swOpenDocOptions\_Silent, "", errors, warnings)

swAssembly = swModel

swModelDocExt = swModel.Extension

swModel.ShowNamedView2("Dimetric", 9)

swModel.ViewZoomtofit2()

swModel.ClearSelection2(True)

'Temporarily fix a component

status = swModelDocExt.SelectByID2("rocker-1@valve\_cam", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)

swAssembly.TemporaryFixGroup()

Stop

'Examine rocker-1@valve\_cam in the FeatureManager design tree and graphics area

'(f)\* and the pink component indicate that the component is fixed

'Press F5

'Changes the component back to floating

swAssembly.TemporaryFixGroupExit()

swModel.ClearSelection2(True)

'Temporarily group components

status = swModelDocExt.SelectByID2("camshaft-1@valve\_cam", "COMPONENT", 0, 0, 0, True, 2, Nothing, 0)

status = swModelDocExt.SelectByID2("rocker-1@valve\_cam", "COMPONENT", 0, 0, 0, True, 2, Nothing, 0)

swAssembly.TemporaryFixGroup()

Stop

'Examine the graphics area

'Pink components indicate that the components are grouped

'Press F5

'Ungroup components

swAssembly.TemporaryFixGroupExit()

End Sub

''' <summary>

''' The SldWorks swApp variable is pre-assigned for you.

''' </summary>

Public swApp As SldWorks

End Class

## Tessellate a Body Example (C#)

```
//-----
// Preconditions:
// 1. Verify that the part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Tessellates the body.
// 2. Examine the graphics area and Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using System;

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Diagnostics;
namespace TessellateABody_CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public SldWorks swApp;
        public void Main()
        {
            int errors = 0;
            int warnings = 0;
            // Get the active document
            ModelDoc2 swModel = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\drive shaft plate.sldprt", (int)swDocumentTypes_e.swDocPART, 0, "",
            ref errors, ref warnings);
            // Get the document title
            Debug.Print(swModel.GetTitle());
            // Cast down to a part
            PartDoc swPart = (PartDoc)swModel;
            // Variant is returned, which can be dealt with as an object
            // Variant represents an array of references
            System.Object vBodies;
            System.Array aBodies;
            // Get bodies
            vBodies = swPart.GetBodies2((int)swBodyType_e.swAllBodies, false);
            // Cast to an array of bodies
            aBodies = (System.Array)vBodies;
            int iNumBodies = aBodies.Length;
            int iNumSolidBodies = 0;
            int iNumSheetBodies = 0;
            // Loop through array
            Body2 swBody;
            for (int b = 0; b < iNumBodies; b++)
            {
                // Get a body and apply cast
                swBody = (Body2)aBodies.GetValue(b);
                // Now call a method on a body
                int nBodyType = swBody.GetType();
                if (nBodyType == (int)swBodyType_e.swSheetBody)
                {
                    iNumSheetBodies++;
                }
                if (nBodyType == (int)swBodyType_e.swSolidBody)
                {
                    iNumSolidBodies++;
                    Face2 swFace = null;
                    Tessellation swTessellation = null;
                    bool bResult = false;
                    // Pass in null so the whole body will be tessellated
                    swTessellation = (Tessellation)swBody.GetTessellation(null);
                    // Set up the Tessellation object
                    swTessellation.NeedFaceFacetMap = true;
                    swTessellation.NeedVertexParams = true;
                    swTessellation.NeedVertexNormal = true;
                    swTessellation.ImprovedQuality = true;
                    Debug.Print("Tessellation is configured for higher-quality data output.");
                }
            }
        }
    }
}
```

```

// How to handle matches across common edges
swTessellation.MatchType = (int)swTessellation.MatchType_e.swTessellation.MatchFacetTopology;

// Do it
bResult = swTessellation.Tessellate();

// Get the number of vertices and facets
Debug.Print("Number of vertices: " + swTessellation.GetVertexCount());
Debug.Print("Number of facets: " + swTessellation.GetFacetCount());

// Now get the facet data per face
int[] aFacetIds;
int iNumFacetIds;

int[] aFinIds;
int[] aVertexIds;
double[] aVertexCoords1;
double[] aVertexCoords2;

// Add sketch for this body
// Speed things up
swModel.SetAddToDB(true);
swModel.SetDisplayWhenAdded(false);
swModel.Insert3DSketch2(false);

// Loop over faces
swFace = (Face2)swBody.GetFirstFace();
while (swFace != null)
{
    aFacetIds = (int[])swTessellation.GetFaceFacets(swFace);
    iNumFacetIds = aFacetIds.Length;
    for (int iFacetIdx = 0; iFacetIdx < iNumFacetIds; iFacetIdx++)
    {
        aFinIds = (int[])swTessellation.GetFacetFins(aFacetIds[iFacetIdx]);
        // There should always be three fins per facet
        for (int iFinIdx = 0; iFinIdx < 3; iFinIdx++)
        {
            aVertexIds = (int[])swTessellation.GetFinVertices(aFinIds[iFinIdx]);
            // Should always be two vertices per fin
            aVertexCoords1 = (double[])swTessellation.GetVertexPoint(aVertexIds[0]);
            aVertexCoords2 = (double[])swTessellation.GetVertexPoint(aVertexIds[1]);
            // Create a line
            swModel.CreateLine2(aVertexCoords1[0], aVertexCoords1[1], aVertexCoords1[2], aVertexCoords2[0], aVertexCoords2[1], aVertexCoords2[2]);
        }
    }
    swFace = (Face2)swFace.GetNextFace();
}

// Close sketch
swModel.Insert3DSketch2(true);

// Clear selection for next pass
swModel.ClearSelection2(true);

// Restore settings
swModel.SetAddToDB(false);
swModel.SetDisplayWhenAdded(true);
}
}
return;
}
}
}

```

## Tessellate a Body Example (VBA)

This example shows how to tessellate a body.

```
'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Tessellates the body.
' 2. Examine the graphics area and Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
```

Option Explicit

```
Sub Main()
    Dim errors As Long
    Dim warnings As Long
    Dim swApp As SldWorks.SldWorks
    Dim swModel As ModelDoc2
    Dim swPart As PartDoc
    Dim b As Integer
    Dim nBodyType As Integer
    Dim swFace As Face2
    Dim swTessellation As Tessellation
    Dim bResult As Boolean
    Dim iNumVertices As Integer
    Dim iNumFacets As Integer
    Dim aFacetIds As Variant
    Dim aFinIds As Variant
    Dim aVertexIds As Variant
    Dim aVertexCoords1 As Variant
    Dim aVertexCoords2 As Variant
    Dim iFacetIdx As Integer
    Dim iFinIdx As Integer
    Dim vBodies As Variant

    Set swApp = Application.SldWorks
    ' Get the active document
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\drive shaft plate.sldprt", swDocumentTypes_e.swDocPART, 0, "", errors, warnings)
    ' Get the document title
    Debug.Print swModel.GetTitle()
    ' Cast down to a part
    Set swPart = swModel
    ' Get bodies
    vBodies = swPart.GetBodies2(swBodyType_e.swAllBodies, False)
    ' Loop through array
    Dim swBody As Body2
    For b = 0 To UBound(vBodies)
        ' Get a body
        Set swBody = vBodies(b)
        ' Now call a method on a body
        nBodyType = swBody.GetType()
        If nBodyType = swBodyType_e.swSolidBody Then
            Set swFace = Nothing
            Set swTessellation = Nothing
            bResult = False
            ' Pass in null so the whole body will be tessellated
            Set swTessellation = swBody.GetTessellation(Empty)
            ' Set up the Tessellation object
            swTessellation.NeedFaceFacetMap = True
            swTessellation.NeedVertexParams = True
            swTessellation.NeedVertexNormal = True
            ' Set option to improve quality of data returned
            swTessellation.ImprovedQuality = True
            Debug.Print ("Tessellation is configured for higher-quality data.")
            ' How to handle matches across common edges
            swTessellation.MatchType = swTessellationMatchType_e.swTessellationMatchFacetTopology
            ' Do the tessellation
            bResult = swTessellation.Tessellate()
```

```

' Get the number of vertices and facets
iNumVertices = swTessellation.GetVertexCount()
Debug.Print ("Vertex count: " & iNumVertices)
iNumFacets = swTessellation.GetFacetCount()
Debug.Print ("Facet count: " & iNumFacets)

' Speed things up
swModel.SetAddToDB (True)
swModel.SetDisplayWhenAdded (False)

' Insert sketch
swModel.Insert3DSketch2 (False)

' Now get the facet data per face
Set swFace = swBody.GetFirstFace()
While Not swFace Is Nothing
    aFacetIds = swTessellation.GetFaceFacets(swFace)
    For iFacetIdx = 0 To UBound(aFacetIds)
        aFinIds = swTessellation.GetFacetFins(aFacetIds(iFacetIdx))
        ' There should always be three fins per facet
        For iFinIdx = 0 To 2
            aVertexIds = swTessellation.GetFinVertices(aFinIds(iFinIdx))
            ' Should always be two vertices per fin
            aVertexCoords1 = swTessellation.GetVertexPoint(aVertexIds(0))
            aVertexCoords2 = swTessellation.GetVertexPoint(aVertexIds(1))

            ' Create a line
            swModel.CreateLine2 aVertexCoords1(0), aVertexCoords1(1), aVertexCoords1(2), aVertexCoords2(0), aVertexCoords2(1), aVertexCoords2(2)
        Next
    Next
    Set swFace = swFace.GetNextFace()
Wend

' Close sketch
swModel.Insert3DSketch2 (True)

' Clear selection for next pass
swModel.ClearSelection2 (True)

' Restore settings
swModel.SetAddToDB (False)
swModel.SetDisplayWhenAdded (True)
End If

Next
Exit Sub
End Sub

```

## Tessellate a Body Example (VB.NET)

This example shows how to tessellate a body.

```
'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Tessellates the body.
' 2. Examine the graphics area and Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub Main()
```

```
        Dim errors As Integer
```

```
        Dim warnings As Integer
```

```
        ' Get the active document
```

```
        Dim swModel As ModelDoc2 = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\drive shaft plate.sldprt", swDocumentTypes_e.swDocPART, 0, "", errors, warnings)
```

```
        ' Get the document title
```

```
        Dim title As [String] = swModel.GetTitle()
```

```
        ' Cast down to a part
```

```
        Dim swPart As PartDoc = DirectCast(swModel, PartDoc)
```

```
        ' Variant is returned, which can be dealt with as an object
```

```
        ' Variant represents an array of references
```

```
        Dim vBodies As System.Object
```

```
        Dim aBodies As System.Array
```

```
        ' Get bodies
```

```
        vBodies = swPart.GetBodies2(CInt(swBodyType_e.swAllBodies), False)
```

```
        ' Cast to an array of bodies
```

```
        aBodies = DirectCast(vBodies, System.Array)
```

```
        Dim iNumBodies As Integer = aBodies.Length
```

```
        Dim iNumSolidBodies As Integer = 0
```

```
        Dim iNumSheetBodies As Integer = 0
```

```
        ' Loop through array
```

```
        Dim swBody As Body2
```

```
        For b As Integer = 0 To iNumBodies - 1
```

```
            ' Get a body and apply cast
```

```
            swBody = DirectCast(aBodies.GetValue(b), Body2)
```

```
            ' Now call a method on a body
```

```
            Dim nBodyType As Integer = swBody.GetType()
```

```
            If nBodyType = CInt(swBodyType_e.swSheetBody) Then
```

```
                iNumSheetBodies += 1
```

```
            End If
```

```
            If nBodyType = CInt(swBodyType_e.swSolidBody) Then
```

```
                iNumSolidBodies += 1
```

```
                Dim swFace As Face2 = Nothing
```

```
                Dim swTessellation As Tessellation = Nothing
```

```
                Dim bResult As Boolean = False
```

```
                ' Pass in null so the whole body will be tessellated
```

```
                swTessellation = DirectCast(swBody.GetTessellation(Nothing), Tessellation)
```

```
                ' Set up the Tessellation object
```

```
                ' Required data
```

```
                swTessellation.NeedFaceFacetMap = True
```

```
                swTessellation.NeedVertexParams = True
```

```
                swTessellation.NeedVertexNormal = True
```

```
                ' Set option to improve quality of data returned
```

```
                swTessellation.ImprovedQuality = True
```

```
                Debug.Print("Tessellation is configured for higher data quality.")
```

```
                ' How to handle matches across common edges
```

```
                swTessellation.MatchType = CInt(swTessellationMatchType_e.swTessellationMatchFacetTopology)
```

```
                ' Do the tessellation
```

```
                bResult = swTessellation.Tessellate()
```

```
                ' Get the number of vertices and facets
```

```

Dim iNumVertices As Integer = swTessellation.GetVertexCount()
Debug.Print("Vertex count: " & iNumVertices)
Dim iNumFacets As Integer = swTessellation.GetFacetCount()
Debug.Print("Facet count: " & iNumFacets)

' Now get the facet data per face
Dim aFacetIds As Integer()
Dim iNumFacetIds As Integer
Dim aFinIds As Integer()
Dim aVertexIds As Integer()
Dim aVertexCoords1 As Double()
Dim aVertexCoords2 As Double()

' Speed things up
swModel.SetAddToDB(True)
swModel.SetDisplayWhenAdded(False)

' Insert sketch
swModel.Insert3DSketch2(False)

' Loop over faces
swFace = DirectCast(swBody.GetFirstFace(), Face2)
While swFace IsNot Nothing

    aFacetIds = DirectCast(swTessellation.GetFaceFacets(swFace), Integer())
    iNumFacetIds = aFacetIds.Length
    For iFacetIdx As Integer = 0 To iNumFacetIds - 1

        aFinIds = DirectCast(swTessellation.GetFacetFins(aFacetIds(iFacetIdx)), Integer())

        ' There should always be three fins per facet
        For iFinIdx As Integer = 0 To 2

            aVertexIds = DirectCast(swTessellation.GetFinVertices(aFinIds(iFinIdx)), Integer())

            ' Should always be two vertices per fin
            aVertexCoords1 = DirectCast(swTessellation.GetVertexPoint(aVertexIds(0)), Double())
            aVertexCoords2 = DirectCast(swTessellation.GetVertexPoint(aVertexIds(1)), Double())

            ' Create a line
            swModel.CreateLine2(aVertexCoords1(0), aVertexCoords1(1), aVertexCoords1(2), aVertexCoords2(0), aVertexCoords2(1), aVertexCoords2(2))

        Next
    Next

    swFace = DirectCast(swFace.GetNextFace(), Face2)
End While

' Close sketch
swModel.Insert3DSketch2(True)

' Clear selection for next pass
swModel.ClearSelection2(True)

' Restore settings
swModel.SetAddToDB(False)
swModel.SetDisplayWhenAdded(True)

End If

Next

Exit Sub

End Sub

Public swApp As SldWorks
End Class

```

## Test for Toolbox Part Example (C#)

This example shows how to test whether a part is a Toolbox part.

```
//-----  
// Preconditions: Open public_documents\samples\tutorial\api\bagel.sldprt.  
//  
// Postconditions: Inspect the Immediate window for the Toolbox part type.  
//  
// NOTE: Because the model is used elsewhere,  
// do not save changes when closing it.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
namespace ToolboxPartType_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        ModelDoc2 part;  
        ModelDocExtension modelDocExt;  
  
        int ret;  
  
        public void Main()  
        {  
            part = (ModelDoc2)swApp.ActiveDoc;  
            modelDocExt = part.Extension;  
            ret = modelDocExt.ToolboxPartType;  
  
            Debug.Print("Toolbox part type as defined in swToolBoxPartType_e? " + ret);  
        }  
  
        public SldWorks swApp;  
    }  
}
```

### Test for Toolbox Part Example (VBA)

This example shows how to test whether a part is a Toolbox part.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\api\bagel.sldprt.  
'  
' Postconditions: Inspect the Immediate window for the Toolbox part type.  
'  
' NOTE: Because the model is used elsewhere,  
' do not save changes when closing it.  
'-----
```

```
Dim swApp As SldWorks.SldWorks  
Dim part As SldWorks.ModelDoc2  
Dim modelDocExt As SldWorks.ModelDocExtension  
Dim ret As Long  
Option Explicit  
Sub main()  
    Set swApp = Application.SldWorks  
    Set part = swApp.ActiveDoc  
    Set modelDocExt = part.Extension  
    ret = modelDocExt.ToolboxPartType  
  
    Debug.Print "Toolbox part type as defined in swToolBoxPartType_e? " & ret  
End Sub
```

### Test for Toolbox Part Example (VB.NET)

This example shows how to test whether a part is a Toolbox part.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\api\bagel.sldprt.  
'  
' Postconditions: Inspect the Immediate window for the Toolbox part type.  
'  
' NOTE: Because the model is used elsewhere,  
' do not save changes when closing it.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim part As ModelDoc2  
    Dim modelDocExt As ModelDocExtension  
    Dim ret As Integer
```

```
    Sub main()
```

```
        part = swApp.ActiveDoc  
        modelDocExt = part.Extension  
        ret = modelDocExt.ToolboxPartType
```

```
        Debug.Print("Toolbox part type as defined in swToolBoxPartType_e? " & ret)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

### Thicken Sheet Body Example (VBA)

This example shows how to thicken a sheet (surface) body.

```
'-----  
' Preconditions:  
' 1. Open a part that contains a sheet body.  
' 2. Select the sheet body in the Surface Bodies folder.  
'  
' Postconditions:  
' 1. Thickens the sheet body and creates a temporary  
'    thickened body.  
' 2. Examine the FeatureManager design tree and  
'    graphics area.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
  
Sub main()  
  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swPart As SldWorks.PartDoc  
    Dim swBody As SldWorks.Body2  
    Dim swFace As SldWorks.Face2  
    Dim swOriginalSheetBody As SldWorks.Body2  
    Dim swThickenedbody As SldWorks.Body2  
    Dim swModeler As SldWorks.Modeler  
    Dim dThickness As Double  
    Dim vSheets As Variant  
    Dim lOptions As Long  
    Dim lErrors As Long  
    Dim lNumSheets As Long  
    Dim aBodies(0) As SldWorks.Body2  
    Dim vBodies As Variant  
    Dim swFeature As SldWorks.Feature  
  
    ' Thickness  
    dThickness = 0.01  
  
    ' Connect to SOLIDWORKS Design  
    Set swApp = Application.SldWorks  
  
    ' Get modeler  
    Set swModeler = swApp.GetModeler  
    Debug.Assert Not swModeler Is Nothing  
  
    ' Get active document  
    Set swModel = swApp.ActiveDoc  
  
    ' Cast down to part  
    Set swPart = swModel  
  
    ' Get the selected sheet body  
    Set swBody = swModel.SelectionManager.GetSelectedObject6(1, -1)  
  
    ' Make a copy, so you can later sew together with other sheets  
    Set swOriginalSheetBody = swBody.Copy  
    Set swThickenedbody = swModeler.ThickenSheet(swOriginalSheetBody, dThickness, swThickenDirection_Side1)  
    Debug.Assert (Not (swThickenedbody Is Nothing))  
    Debug.Assert (swThickenedbody.GetType = swBodyType_e.swSolidBody)  
    Set aBodies(0) = swThickenedbody  
    vBodies = aBodies  
  
    ' Turn temporary body into a feature  
    Set swFeature = swPart.CreateFeatureFromBody3(vBodies(0), False, swCreateFeatureBodyOpts_e.swCreateFeatureBodyCheck)  
  
    swModel.EditRebuild3  
  
End Sub
```

## Thicken Surface and Generate Boss Example (C#)

This example shows how to thicken a surface and then generate a boss.

```
//-----
// Preconditions:
// 1. Open a model document and select a boundary surface.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates a thicken feature.
// 2. Inspect the Immediate window.
//-----

using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetThickenFeatureData_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            FeatureManager swFeatMgr = default(FeatureManager);
            Feature swFeat = default(Feature);
            ThickenFeatureData swThicken = default(ThickenFeatureData);
            bool bRet = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeatMgr = swModel.FeatureManager;

            bRet = swSelMgr.SetSelectedObjectMark(1, 1, (int)swSelectionMarkAction_e.swSelectionMarkSet);

            swFeat = swFeatMgr.FeatureBossThicken(0.001, (int)swThickenThicknessType_e.swThickenSideOne, 0, false, true, true, true);
            swThicken = (ThickenFeatureData)swFeat.GetDefinition();

            Debug.Print("Feature: " + swFeat.Name);
            Debug.Print(" Automatically select all bodies? " + swThicken.AutoSelect);
            Debug.Print(" True if thicken feature affects specified bodies, false if it affects all bodies: " + swThicken.FeaturesSelected);
            Debug.Print(" Is a boss feature? " + swThicken.IsBossFeature());
            Debug.Print(" Merge the results if a multibody part? " + swThicken.Merge);
            Debug.Print(" Fill a volume? " + swThicken.FillVolume);
            Debug.Print(" Thickness: " + swThicken.Thickness * 1000.0 + " mm");
            Debug.Print(" Thickness type as defined in swThickenThicknessType_e: " + swThicken.ThicknessSide);

        }

        public SldWorks swApp;
    }
}
```

### Thicken Surface and Generate Boss Example (VBA)

This example shows how to thicken a surface and then generate a boss.

```
'-----  
' Preconditions:  
' 1. Open a model document and select a boundary surface.  
' 2. Open an Immediate window.  
'  
' Postconditions:  
' 1. Creates a thicken feature.  
' 2. Inspect the Immediate window.  
'-----
```

Option Explicit

```
Sub main()  
    Dim swApp                As SldWorks.SldWorks  
    Dim swModel              As SldWorks.ModelDoc2  
    Dim swSelMgr              As SldWorks.SelectionMgr  
    Dim swFeatMgr             As SldWorks.FeatureManager  
    Dim swFeat               As SldWorks.Feature  
    Dim swThicken             As SldWorks.ThickenFeatureData  
    Dim bRet                  As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeatMgr = swModel.FeatureManager  
  
    bRet = swSelMgr.SetSelectedObjectMark(1, 1, swSelectionMarkSet)  
  
    Set swFeat = swFeatMgr.FeatureBossThicken(0.001, swThickenSideOne, 0, False, True, True, True)  
    Set swThicken = swFeat.GetDefinition  
    Debug.Print "Feature: " & swFeat.Name  
    Debug.Print " Automatically select all bodies? " & swThicken.AutoSelect  
    Debug.Print " True if thicken feature affects specified bodies, false if it affects all bodies: " & swThicken.FeatureScope  
    Debug.Print " Is a boss feature? " & swThicken.IsBossFeature  
    Debug.Print " Merge the results if a multibody part? " & swThicken.Merge  
    Debug.Print " Fill a volume? " & swThicken.FillVolume  
    Debug.Print " Thickness: " & swThicken.Thickness * 1000# & " mm"  
    Debug.Print " Thickness type as defined in swThickenThicknessType_e: " & swThicken.ThicknessSide  
End Sub
```

## Thicken Surface and Generate Boss Example (VB.NET)

This example shows how to thicken a surface and then generate a boss..

```

'-----
' Preconditions:
' 1. Open a model document and select a boundary surface.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a thicken feature.
' 2. Inspect the Immediate window.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swFeatMgr As FeatureManager
        Dim swFeat As Feature
        Dim swThicken As ThickenFeatureData
        Dim bRet As Boolean

```

```

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swFeatMgr = swModel.FeatureManager

```

```
        bRet = swSelMgr.SetSelectedObjectMark(1, 1, swSelectionMarkAction_e.swSelectionMarkSet)
```

```

        swFeat = swFeatMgr.FeatureBossThicken(0.001, swThickenThicknessType_e.swThickenSideOne, 0, False, True, True, True)
        swThicken = swFeat.GetDefinition

```

```

        Debug.Print("Feature: " & swFeat.Name)
        Debug.Print("  Automatically select all bodies? " & swThicken.AutoSelect)
        Debug.Print("  True if thicken feature affects specified bodies, false if it affects all bodies: " & swThicken.FeatureScope)
        Debug.Print("  Is a boss feature? " & swThicken.IsBossFeature)
        Debug.Print("  Merge the results if a multibody part? " & swThicken.Merge)
        Debug.Print("  Fill a volume? " & swThicken.FillVolume)
        Debug.Print("  Thickness: " & swThicken.Thickness * 1000.0# & " mm")
        Debug.Print("  Thickness type as defined in swThickenThicknessType_e: " & swThicken.ThicknessSide)

```

```
    End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

### Toggle Display of Surface Curvatures Example (VBA)

This example shows how to toggle the display of surface curvatures.

```
'-----  
' Preconditions: Open a part.  
'  
' Postconditions:  
' 1. Turns the display of the surface  
'   curvatures on if off or off if on.  
' 2. Examine the graphics area.  
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2
```

```
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc
```

```
    swModel.ViewDisplayCurvature
```

End Sub

## Transform Coordinates from Sketch to Model Space Example (VBA)

When an entity is selected while editing a sketch, for example a sketch point, the coordinates of the point are shown in the space of the sketch. Sometimes it is desirable to know the coordinates of this point in the space of the model.

This example shows how to use various transforms and math utilities to transform a point in sketch space to model space.

```
'-----
'
' Preconditions:
' (1) Part, assembly or drawing is open.
' (2) If a part or assembly, then a sketch is being edited.
' (3) If a part or assembly, then an entity is selected in
' the sketch.
' (4) If a drawing, then an entity is selected.
'
' Postconditions: None
'
' NOTES:
' (1) If the sketch is a 3D sketch, then the selected sketch
' point is automatically in model coordinates.
' (2) If the sketch is a 3D sketch, then its transform is the
' unit transform.
'
'-----

Option Explicit
Public Function GetModelCoordinates _
( _
    swApp As SldWorks.SldWorks, _
    swSketch As SldWorks.sketch, _
    vPtArr As Variant _
) As Variant
    Dim swMathPt As SldWorks.MathPoint
    Dim swMathUtil As SldWorks.MathUtility
    Dim swMathTrans As SldWorks.MathTransform
    Set swMathUtil = swApp.GetMathUtility
    Set swMathPt = swMathUtil.CreatePoint(vPtArr)
    ' Is a unit transform if 3D sketch; for example, selected sketch
    ' point is automatically in model space
    Set swMathTrans = swSketch.ModelToSketchTransform
    Set swMathTrans = swMathTrans.Inverse
    Set swMathPt = swMathPt.MultiplyTransform(swMathTrans)
    GetModelCoordinates = swMathPt.ArrayData
End Function

Sub main()
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swSketch As SldWorks.sketch
    Dim vSketchSelPt As Variant
    Dim vModelSelPt As Variant
    Dim i As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swSketch = swModel.GetActiveSketch2

    vSketchSelPt = swSelMgr.GetSelectionPointInSketchSpace(1)
    vModelSelPt = GetModelCoordinates(swApp, swSketch, vSketchSelPt)

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " Is3D sketch = " & swSketch.Is3D
    Debug.Print " SelPt (sketch space) = (" & vSketchSelPt(0) * 1000# & ", " & vSketchSelPt(1) * 1000# & ", " & vSketchSelPt(2) * 1000# & ") mm"
    Debug.Print " SelPt (model space) = (" & vModelSelPt(0) * 1000# & ", " & vModelSelPt(1) * 1000# & ", " & vModelSelPt(2) * 1000# & ") mm"
End Sub
'-----
```

## Transform Point from Component Space to Assembly Space Example (C#)

This example shows how to transform a point from component space to assembly space.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly document to open
//    exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified assembly document.
// 2. Selects a component.
// 3. Transforms the component's origin to a point in
//    assembly space.
// 4. Examine the Immediate window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Transform2.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            MathUtility swMathUtil = default(MathUtility);
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Component2 swComp = default(Component2);
            MathTransform swXform = default(MathTransform);
            double[] nPt = new double[3];
            object vPt = null;
            MathPoint swPt = default(MathPoint);
            bool bRet = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;

            // Open assembly
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\smartcomponents\\stepped_shaft.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref e
            rrors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            bRet = swModelDocExt.SelectByID2("stepped_shaft-1@stepped_shaft", "COMPONENT", 0, 0, 0, false, 0, null, 0);

            swMathUtil = (MathUtility)swApp.GetMathUtility();
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swComp = (Component2)swSelMgr.GetSelectedObjectsComponent(1);
            swXform = (MathTransform)swComp.Transform2;

            // Point at component origin
            nPt[0] = 0.0;
            nPt[1] = 0.0;
            nPt[2] = 0.0;
            vPt = nPt;
            swPt = (MathPoint)swMathUtil.CreatePoint(vPt);
            swPt = (MathPoint)swPt.MultiplyTransform(swXform);
            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" Component = " + swComp.Name2 + " [" + swComp.GetPathName() + "]");
            Debug.Print(" Point in component = (" + nPt[0] * 1000.0 + ", " + nPt[1] * 1000.0 + ", " + nPt[2] * 1000.0 + ") mm");
            Debug.Print(" Point in assembly = (" + ((double[])swPt.ArrayData)[0] * 1000.0 + ", " + ((double[])swPt.ArrayData)
            [1] * 1000.0 + ", " + ((double[])swPt.ArrayData)[2] * 1000.0 + ") mm");

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

## Transform Point from Component Space to Assembly Space Example (VBA)

This example shows how to transform a point from component space to assembly space.

```
'-----
' Preconditions:
' 1. Verify that the specified assembly document to open
'    exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly document.
' 2. Selects a component.
' 3. Transforms the component's origin to a point in
'    assembly space.
' 4. Examine the Immediate window.
'-----
```

Option Explicit

Sub main()

```
Dim swApp As SldWorks.SldWorks
Dim swMathUtil As SldWorks.MathUtility
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swComp As SldWorks.Component2
Dim swXform As SldWorks.MathTransform
Dim nPt(2) As Double
Dim vPt As Variant
Dim swPt As SldWorks.MathPoint
Dim bRet As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String

Set swApp = Application.SldWorks

' Open assembly
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\smartcomponents\stepped_shaft.sldasm"
Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
Set swModelDocExt = swModel.Extension
bRet = swModelDocExt.SelectByID2("stepped_shaft-1@stepped_shaft", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)

Set swMathUtil = swApp.GetMathUtility
Set swSelMgr = swModel.SelectionManager
Set swComp = swSelMgr.GetSelectedObjectsComponent(1)
Set swXform = swComp.Transform2

' Point at component origin
nPt(0) = 0#
nPt(1) = 0#
nPt(2) = 0#
vPt = nPt
Set swPt = swMathUtil.CreatePoint(vPt)
Set swPt = swPt.MultiplyTransform(swXform)

Debug.Print "File = " & swModel.GetPathName
Debug.Print "  Component = " & swComp.Name2 & " [" & swComp.GetPathName & "]"
Debug.Print "    Point in component = (" & nPt(0) * 1000# & ", " & nPt(1) * 1000# & ", " & nPt(2) * 1000# & ") mm"
Debug.Print "      Point in assembly = (" & swPt.ArrayData(0) * 1000# & ", " & swPt.ArrayData(1) * 1000# & ", " & swPt.ArrayData(2) * 1000# & ") mm"
```

End Sub

## Transform Point from Component Space to Assembly Space Example (VB.NET)

This example shows how to transform a point from component space to assembly space.

```

-----
Preconditions:
1. Verify that the specified assembly document to open exists.
2. Open the Immediate window.
Postconditions:
1. Opens the specified assembly document.
2. Selects a component.
3. Transforms the component's origin to a point in assembly space.
4. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swMathUtil As MathUtility
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swComp As Component2
        Dim swXform As MathTransform
        Dim nPt(2) As Double
        Dim vPt As Object
        Dim swPt As MathPoint
        Dim bRet As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        ' Open assembly
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\smartcomponents\stepped_shaft.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
        bRet = swModelDocExt.SelectByID2("stepped_shaft-1@stepped_shaft", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)

        swMathUtil = swApp.GetMathUtility
        swSelMgr = swModel.SelectionManager
        swComp = swSelMgr.GetSelectedObjectsComponent(1)
        swXform = swComp.Transform2

        ' Point at component origin
        nPt(0) = 0.0#
        nPt(1) = 0.0#
        nPt(2) = 0.0#
        vPt = nPt
        swPt = swMathUtil.CreatePoint(vPt)
        swPt = swPt.MultiplyTransform(swXform)
        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print("    Component = " & swComp.Name2 & " [" & swComp.GetPathName & "]")
        Debug.Print("    Point in component = (" & nPt(0) * 1000.0# & ", " & nPt(1) * 1000.0# & ", " & nPt(2) * 1000.0# & ") mm")
        Debug.Print("    Point in assembly = (" & swPt.ArrayData(0) * 1000.0# & ", " & swPt.ArrayData(1) * 1000.0# & ", " & swPt.ArrayData(2) * 1000.0# &
") mm")

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

## Transform Sketch to Model Example (VBA)

When a sketch point is created, its x, y, and z location values are in relation to the sketch origin. This example shows how to get the sketch point's coordinates in relation to the model origin using a MathUtility object.

```

'-----
' Preconditions:
' 1. Open a part.
' 2. Select a sketch.
'
' Postconditions:
' 1. Gets a MathUtility object from the current model document.
' 2. Gets the selected sketch that contains the points whose
'   coordinates to modify.
' 3. Fills array with all of the points in the sketch.
' 4. Creates a coordinate array with the x, y, and z value sof a sketch point
'   from the sketch-point array.
' 5. Creates a new MathPoint object from the MathUtility object, providing
'   the coordinate array for the location of the MathPoint.
' 6. Gets and displays the model-to-sketch transform for this sketch.
' 7. Click OK to close the message box.
' 8. Calls IMathTransform::Inverse, which provides a MathTransform
'   object from the sketch coordinates to the model coordinates.
' 9. Calls IMathPoint::MulitplyTransform(MathTransform) to move
'   the MathPoint object into the model.
' 10. Displays the point coordinates in relation to the model origin.
' 11. Click OK to close the message box.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim selMgr As SldWorks.SelectionMgr
Dim Model As SldWorks.ModelDoc2
Dim SketchPoints As Variant
Dim SketchFeature As SldWorks.Feature
Dim PointCoords(2) As Double
Dim MathUtil As SldWorks.MathUtility
Dim MathTrans As SldWorks.MathTransform
Dim MathP As SldWorks.MathPoint
Dim ModelSketchTransform As Variant

Sub main()

'Connect to SOLIDWORKS Design
Set swApp = CreateObject("SldWorks.Application")
Set Model = swApp.ActiveDoc

'Prepare the MathUtility
Set MathUtil = swApp.GetMathUtility

'Get the SelectionMgr
Set selMgr = Model.SelectionManager

'Get the sketch from the SelectionMgr
Set SketchFeature = selMgr.GetSelectedObject6(1, 0)
Set SketchFeature = SketchFeature.GetSpecificFeature2

'Get the sketch points
SketchPoints = SketchFeature.GetSketchPoints2

'Build a coordinate array from the first point in the sketch
PointCoords(0) = SketchPoints(0).X
PointCoords(1) = SketchPoints(0).Y
PointCoords(2) = SketchPoints(0).Z

'Create the new MathPoint from the sketch point data
'MathP refers to the point location in the sketch coordinates
Set MathP = MathUtil.CreatePoint(PointCoords)

'Display the point coordinates in relation to the sketch origin
SketchPoints = MathP.ArrayData
MsgBox "Point coordinates in relation to the sketch origin: " & SketchPoints(0) & ", " & SketchPoints(1) & ", " & SketchPoints(2)

'Get the model-to-sketch transform for this sketch
Set MathTrans = SketchFeature.ModelToSketchTransform

'Get the inversion of the transform
Set MathTrans = MathTrans.Inverse

'Multiply the point by the inverse transform
'MathP now refers to the point location in the model coordinates
Set MathP = MathP.MultiplyTransform(MathTrans)

'Display the point coordinates in relation to the model origin
SketchPoints = MathP.ArrayData
MsgBox "Point coordinates in relation to the model origin: " & SketchPoints(0) & ", " & SketchPoints(1) & ", " & SketchPoints(2)

End Sub

```

## Translate Move Face Feature Example (C#)

This example shows how to translate a Move Face feature.

```
//-----
// Preconditions:
// 1. Verify that the specified document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates Move Face1 in the FeatureManager design tree.
// 2. Modifies the translation parameters of Move Face1.
// 3. Gets the type of end condition and offset distance of Move Face1.
// 4. Examine the Immediate window.
//
// NOTE: Because the model document is used elsewhere,
// do not save changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

Namespace IMoveFaceFeatureDataCSharp.csproj
{
    Partial Class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        FeatureManager swFeatMgr;
        Feature swFeat;
        MoveFaceFeatureData swMoveFaceFeat;
        object transParams;
        bool boolstatus;
        double[] triadParams = new double[3];
        int fileerror;
        int filewarning;

        public void Main()
        {
            // Open the document
            swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\assemblymates\\knee.sldprt", (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref fileerror
, ref filewarning);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;
            swFeatMgr = swModel.FeatureManager;

            // Translation parameters
            triadParams[0] = 0;
            triadParams[1] = 0.05;
            triadParams[2] = 0;

            transParams = triadParams;

            // Select face to move
            boolstatus = swModelDocExt.SelectByID("", "FACE", 0.04239074672171, 0.015874999999999, 0.3283508339712, true, 1, null, 0);

            // Create the Move Face feature by
            // translating the selected face
            swFeat = (Feature)swFeatMgr.InsertMoveFace3((int)swMoveFaceType_e.swMoveFaceTypeTranslate, false, 0, 0, (transParams), null, (int)swEndCondi
ons_e.swEndCondBlind, 0);

            // Modify the Move Face feature
            swMoveFaceFeat = (MoveFaceFeatureData)swFeat.GetDefinition();

            // Roll back the Move Face feature
            swMoveFaceFeat.AccessSelections(swModel, null);

            triadParams[0] = 0;
            triadParams[1] = 0.1;
            triadParams[2] = 0;

            transParams = triadParams;

            swMoveFaceFeat.TriadTranslationParameters = (transParams);

            // Get type of end condition and offset distance from which the Move Face feature is translated
            Debug.Print("Type of end condition to which the Move Face feature is translated: " + swMoveFaceFeat.EndCondition);

            if (swMoveFaceFeat.EndCondition == 5)
            {
                Debug.Print("Offset distance of the Move Face feature: " + (swMoveFaceFeat.OffsetDistance * 39.37) + " inches");
            }
            else
            {
                Debug.Print("Offset distance of the Move Face feature: " + (swMoveFaceFeat.Distance * 39.37) + " inches");
            }

            // Roll back the part with the modified Move Face feature
            swFeat.ModifyDefinition(swMoveFaceFeat, swModel, null);
        }
    }
}
```

```
    public SldWorks swApp;  
}
```

## Translate Move Face Feature Example (VBA)

This example shows how to translate a Move Face feature.

```
'-----
' Preconditions:
' 1. Verify that the specified document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates Move Facel in the FeatureManager design tree.
' 2. Modifies the translation parameters of Move Facel.
' 3. Gets the type of end condition and offset distance of Move Facel.
' 4. Examine the Immediate window.
'
' NOTE: Because the model document is used elsewhere,
' do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatMgr As SldWorks.FeatureManager
Dim swFeat As SldWorks.Feature
Dim swMoveFaceFeat As SldWorks.MoveFaceFeatureData
Dim transParams As Variant
Dim boolstatus As Boolean
Dim triadParams(0 To 2) As Double
Dim fileerror As Long
Dim filewarning As Long

Sub main()

    Set swApp = Application.SldWorks

    ' Open the SOLIDWORKS Design document
    swApp.OpenDoc6 "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\assemblymates\knee.sldprt", swDocPART, swOpenDocOptions_Silent,
    "", fileerror, filewarning
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swFeatMgr = swModel.FeatureManager

    ' Translation parameters
    triadParams(0) = 0
    triadParams(1) = 0.05
    triadParams(2) = 0

    transParams = triadParams

    ' Select face to move
    boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.04239074672171, 0.0158749999999999, 0.3283508339712, False, 1, Nothing, 0)

    ' Create the Move Face feature by
    ' translating the selected face
    Set swFeat = swFeatMgr.InsertMoveFace3(swMoveFaceTypeTranslate, False, 0, 0, (transParams), Nothing, swEndCondBlind, 0)

    ' Modify the Move Face feature
    Set swMoveFaceFeat = swFeat.GetDefinition

    ' Roll back the Move Face feature
    swMoveFaceFeat.AccessSelections swModel, Nothing

    triadParams(0) = 0
    triadParams(1) = 0.1
    triadParams(2) = 0

    transParams = triadParams

    swMoveFaceFeat.TriadTranslationParameters = (transParams)

    ' Get type of end condition and offset distance from which the Move Face feature is translated
    Debug.Print ("Type of end condition to which the Move Face feature is translated: " & swMoveFaceFeat.EndCondition)

    If swMoveFaceFeat.EndCondition = 5 Then
        Debug.Print ("Offset distance of the Move Face feature: " & (swMoveFaceFeat.OffsetDistance * 39.37) & " inches")
    Else
        Debug.Print ("Offset distance of the Move Face feature: " & (swMoveFaceFeat.Distance * 39.37) & " inches")
    End If

    ' Roll back the part with the modified Move Face feature
    swFeat.ModifyDefinition swMoveFaceFeat, swModel, Nothing

End Sub
```

## Translate Move Face Feature Example (VB.NET)

This example shows how to translate a Move Face feature.

```

-----
Preconditions:
1. Verify that the specified document to open exists.
2. Open the Immediate window.
Postconditions:
1. Creates Move Face1 in the FeatureManager design tree.
2. Modifies the translation parameters of Move Face1.
3. Gets the type of end condition and offset distance of Move Face1.
4. Examine the Immediate window.
NOTE: Because the model document is used elsewhere,
do not save changes.
-----
Imports System.Collections
Imports System.Collections.Generic
Imports System.Data
Imports System.Diagnostics
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swFeatMgr As FeatureManager
    Dim swFeat As Feature
    Dim swMoveFaceFeat As MoveFaceFeatureData
    Dim transParams As Object
    Dim boolstatus As Boolean
    Dim triadParams As Double() = New Double(2) {}
    Dim fileerror As Integer
    Dim filewarning As Integer

    Public Sub Main()
        ' Open the document
        swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\assemblymates\knee.sldprt", swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", fileerror, filewarning)
        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swFeatMgr = swModel.FeatureManager

        ' Translation parameters
        triadParams(0) = 0
        triadParams(1) = 0.05
        triadParams(2) = 0

        transParams = triadParams

        ' Select face to move
        boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.04239074672171, 0.015874999999999, 0.3283508339712, True, 1, Nothing, 0)

        ' Create the Move Face feature by
        ' translating the selected face
        swFeat = swFeatMgr.InsertMoveFace3(swMoveFaceType_e.swMoveFaceTypeTranslate, False, 0, 0, transParams, Nothing, swEndConditions_e.swEndCondBlind,
0)

        ' Modify the Move Face feature
        swMoveFaceFeat = swFeat.GetDefinition

        ' Roll back the Move Face feature
        swMoveFaceFeat.AccessSelections(swModel, Nothing)

        triadParams(0) = 0
        triadParams(1) = 0.1
        triadParams(2) = 0

        transParams = triadParams

        swMoveFaceFeat.TriadTranslationParameters = (transParams)

        ' Get type of end condition and offset distance from which the Move Face feature is translated
        Debug.Print("Type of end condition to which the Move Face feature is translated: " & swMoveFaceFeat.EndCondition)

        If swMoveFaceFeat.EndCondition = 5 Then
            Debug.Print("Offset distance of the Move Face feature: " & (swMoveFaceFeat.OffsetDistance * 39.37) & " inches")
        Else
            Debug.Print("Offset distance of the Move Face feature: " & (swMoveFaceFeat.Distance * 39.37) & " inches")
        End If

        ' Roll back the part with the modified Move Face feature
        swFeat.ModifyDefinition(swMoveFaceFeat, swModel, Nothing)

    End Sub

    Public swApp As SldWorks

End Class

```

## Translate Sketch Example (C#)

This example shows how to move a sketch.

```

-----
' Preconditions: Verify that the specified template exists.
'
' Postconditions:
' 1. Creates a sketch.
' 2. Creates a parabola.
' 3. While observing the graphics area, press F5 at
'    System.Diagnostics.Debugger.Break to move the sketch.
-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System.Windows.Forms;
namespace TranslateSketch_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2017\\templates\\Part.prtdot", 0, 0,
0);

            if (swModel == null)
            {
                swApp.SendMsgToUser2("A part document must be active.", (int)swMessageBoxIcon_e.swMbWarning,
(int)swMessageBoxBtn_e.swMbOk);
                return;
            }

            int modelType = 0;
            modelType = swModel.GetType();

            if (modelType != (int)swDocumentTypes_e.swDocPART)
            {
                swApp.SendMsgToUser2("A part document must be active.", (int)swMessageBoxIcon_e.swMbWarning,
(int)swMessageBoxBtn_e.swMbOk);
                return;
            }

            //Select a plane on which to sketch
            if (SelectPlane(swModel) == false)
            {
                MessageBox.Show("Could not select plane.");
                return;
            }

            //Get point data
            SketchPoint pFocal = default(SketchPoint);
            SketchPoint pApex = default(SketchPoint);
            SketchPoint pStart = default(SketchPoint);
            SketchPoint pEnd = default(SketchPoint);
            SketchManager swSkMgr = default(SketchManager);
            swSkMgr = swModel.SketchManager;

            SelectionMgr swSelMgr = default(SelectionMgr);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            Sketch swSketch = default(Sketch);
            swSkMgr.InsertSketch(true);
            swSketch = swSkMgr.ActiveSketch;

            // Focal point
            pFocal = swSkMgr.CreatePoint(0, -0.025930732990048, 0);
            // Apex point
            pApex = swSkMgr.CreatePoint(0.0110754938634727, -0.0485199777778575, 0);
            // Start point
            pStart = swSkMgr.CreatePoint(0.057136404168939, 0.0869770346454566, 0);
            // End point
            pEnd = swSkMgr.CreatePoint(-0.120690397734139, -0.00465528735997846, 0);

            object[] vPoint = null;

            // Make sure a sketch is active
            if (swSketch == null)
            {
                MessageBox.Show("Please sketch a focal point, apex point, start point, and end point.");
                return;
            }

```

```

    }

    vPoint = (object[])swSketch.GetSketchPoints2();

    SketchParabola SkParabola = default(SketchParabola);
    SkParabola = (SketchParabola)swModel.SketchManager.CreateParabola(pFocal.X, pFocal.Y, 0, pApex.X, pApex.Y, 0, pStart.X,
pStart.Y, 0, pEnd.X,
pEnd.Y, 0);

    swModel.ViewZoomtofit2();
    swSkMgr.InsertSketch(true);

    System.Diagnostics.Debugger.Break();

    swModel.SketchModifyTranslate(pApex.X, pApex.Y, 0.06, -0.01);
}

public bool SelectPlane(ModelDoc2 Plane)
{
    bool functionReturnValue = false;

    Feature featureTemp = default(Feature);
    featureTemp = (Feature)Plane.FirstFeature();

    while ((featureTemp != null))
    {
        string sFeatureName = null;
        sFeatureName = featureTemp.GetTypeName2();

        if (sFeatureName == "RefPlane")
        {
            featureTemp.Select2(true, 0);
            functionReturnValue = true;
            return functionReturnValue;
        }

        featureTemp = (Feature)featureTemp.GetNextFeature();
    }
    return functionReturnValue;
}

public SldWorks swApp;
}
}

```

## Translate Sketch Example (VBA)

This example shows how to move a sketch.

```
'-----
' Preconditions: Verify that the specified template exists.
'
' Postconditions:
' 1. Creates a sketch.
' 2. Creates a parabola.
' 3. While observing the graphics area, press F5 at Stop
'    to move the sketch.
'-----
```

Option Explicit

```
Sub main()
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2

    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2017\templates\Part.prtdot", 0, 0, 0)

    If swModel Is Nothing Then
        swApp.SendMsgToUser2 "A part document must be active.", swMbWarning, swMbOk
        Exit Sub
    End If

    Dim modelType As Long
    modelType = swModel.GetType

    If modelType <> SwConst.swDocPART Then
        swApp.SendMsgToUser2 "A part document must be active.", swMbWarning, swMbOk
        Exit Sub
    End If

    'Select a plane on which to sketch
    If SelectPlane(swModel) = False Then
        MsgBox "Could not select plane."
        Exit Sub
    End If

    'Get point data
    Dim pFocal As SldWorks.SketchPoint
    Dim pApex As SldWorks.SketchPoint
    Dim pStart As SldWorks.SketchPoint
    Dim pEnd As SldWorks.SketchPoint
    Dim swSkMgr As SldWorks.SketchManager
    Set swSkMgr = swModel.SketchManager

    Dim swSelMgr As SldWorks.SelectionMgr
    Set swSelMgr = swModel.SelectionManager

    Dim swSketch As Sketch
    swSkMgr.InsertSketch True
    Set swSketch = swSkMgr.ActiveSketch

    ' Focal point
    Set pFocal = swSkMgr.CreatePoint(0, -0.025930732990048, 0)
    ' Apex point
    Set pApex = swSkMgr.CreatePoint(1.10754938634727E-02, -4.85199777778575E-02, 0)
    ' Start point
    Set pStart = swSkMgr.CreatePoint(0.057136404168939, 8.69770346454566E-02, 0)
    ' End point
    Set pEnd = swSkMgr.CreatePoint(-0.120690397734139, -4.65528735997846E-03, 0)

    Dim vPoint As Variant

    ' Make sure a sketch is active
    If swSketch Is Nothing Then
        MsgBox "Please sketch a focal point, apex point, start point, and end point."
        Exit Sub
    End If

    vPoint = swSketch.GetSketchPoints2

    Dim SkParabola As SldWorks.SketchParabola
    Set SkParabola = swModel.SketchManager.CreateParabola(pFocal.X, pFocal.Y, 0, pApex.X, pApex.Y, 0, pStart.X, pStart.Y, 0,
    pEnd.X, pEnd.Y, 0)

    swModel.ViewZoomtofit2
    swSkMgr.InsertSketch True
End Sub
```

Stop

swModel.**SketchModifyTranslate** pApex.X, pApex.Y, 0.06, -0.01

End Sub

Public Function SelectPlane(Plane As SldWorks.ModelDoc2) As Boolean

Dim featureTemp As Feature

Set featureTemp = Plane.**FirstFeature**

While Not featureTemp Is Nothing

Dim sFeatureName As String

sFeatureName = featureTemp.**GetTypeNames**

If sFeatureName = "RefPlane" Then

featureTemp.**Select2** True, 0

SelectPlane = True

Exit Function

End If

Set featureTemp = featureTemp.**GetNextFeature**

Wend

End Function

## Translate Sketch Example (VB.NET)

This example shows how to move a sketch.

```
'-----
' Preconditions: Verify that the specified template exists.
'
' Postconditions:
' 1. Creates a sketch.
' 2. Creates a parabola.
' 3. While observing the graphics area, press F5 at Stop
'    to move the sketch.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Sub main()

        Dim swModel As ModelDoc2

        swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS
2012\templates\Part.prtdot", 0, 0, 0)

        If swModel Is Nothing Then

            swApp.SendMsgToUser2("A
part document must be active.", swMessageBoxIcon_e.swMbWarning, swMessageBoxBtn_e.swMbOk)

            Exit Sub

        End If

        Dim modelType As Integer

        modelType = swModel.GetType

        If modelType <> swDocumentTypes_e.swDocPART Then

            swApp.SendMsgToUser2("A
part document must be active.", swMessageBoxIcon_e.swMbWarning, swMessageBoxBtn_e.swMbOk)

            Exit Sub

        End If

        'Select
a plane on which to sketch

        If SelectPlane(swModel) = False Then

            MsgBox("Could not select plane.")

            Exit Sub

        End If

        'Get
point data

        Dim pFocal As SketchPoint

        Dim pApex As SketchPoint

        Dim pStart As SketchPoint

        Dim pEnd As SketchPoint

        Dim swSkMgr As SketchManager

        swSkMgr = swModel.SketchManager

        Dim swSelMgr As SelectionMgr

        swSelMgr = swModel.SelectionManager

        Dim swSketch As Sketch
```

```

        swSkMgr.InsertSketch(True)

        swSketch = swSkMgr.ActiveSketch

        ' Focal point
        pFocal = swSkMgr.CreatePoint(0,
-0.025930732990048, 0)

        ' Apex point
        pApex = swSkMgr.CreatePoint(0.0110754938634727,
-0.048519977778575, 0)

        ' Start point
        pStart = swSkMgr.CreatePoint(0.057136404168939,
0.0869770346454566, 0)

        ' End point
        pEnd = swSkMgr.CreatePoint(-0.120690397734139,
-0.00465528735997846, 0)

        Dim vPoint As Object

        '
Make sure a sketch is active
        If swSketch Is Nothing Then
            MsgBox("Please sketch a focal point, apex point, start point, and end point.")
            Exit Sub
        End If

        vPoint = swSketch.GetSketchPoints2

        Dim SkParabola As SketchParabola
        SkParabola = swModel.SketchManager.CreateParabola(pFocal.X, pFocal.Y, 0, pApex.X, pApex.Y, 0, pStart.X, pStart.Y, 0, pEnd.X, pEnd.Y, 0)

        swModel.ViewZoomtofit2()
        swSkMgr.InsertSketch(True)

        Stop

        swModel.SketchModifyTranslate(pApex.X, pApex.Y, 0.06, -0.01)

    End Sub

    Public Function SelectPlane(ByVal Plane As ModelDoc2) As Boolean

        Dim featureTemp As Feature
        featureTemp = Plane.FirstFeature

        While Not featureTemp Is Nothing
            Dim sFeatureName As String
            sFeatureName = featureTemp.GetTypeName2

            If sFeatureName = "RefPlane" Then
                featureTemp.Select2(True,
0)

                SelectPlane = True
                Exit Function
            End If

            featureTemp = featureTemp.GetNextFeature

        End While

    End Function

```

Public swApp As StdWorks

End Class

### Trap Error When Closing PropertyManager Page Example (VB.NET)

This example shows how to cancel the closing of a PropertyManager page if your application detects an error.

```
...
Sub OnClose(ByVal reason As Integer) Implements PropertyManagerPage2Handler4.OnClose
    ' This function must contain code, even if it does nothing, to prevent the
    ' .NET runtime environment from doing garbage collection at the wrong time.
    Dim IndentSize As Integer
    IndentSize = System.Diagnostics.Debug.IndentSize
    System.Diagnostics.Debug.WriteLine(IndentSize)

    If reason = SwConst.swPropertyManagerPageCloseReasons_e.swPropertyManagerPageClose_Okay Then
        Dim e As New System.Runtime.InteropServices.COMException("cancel close", 1)

        Throw e
    End If
End Sub
...
```

## Traverse All Cosmetic Threads Example (C#)

This example shows how to traverse all cosmetic threads in a part and extract their data.

**NOTE:** In a part or assembly, a cosmetic thread is a subfeature of a hole or cut extrusion. Thus, you can traverse all of the cosmetic threads in a model using the IFeature traversal methods.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\holecube.sldprt.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates a Helicoil Metric standard cosmetic thread.
// 2. Examine the Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace InsertCosmeticThread3_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            Feature swFeat = default(Feature);
            Feature swSubFeat = default(Feature);
            string sFeatType = null;
            CosmeticThreadFeatureData swCosThread = default(CosmeticThreadFeatureData);
            bool bRet = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;

            bRet = swModel.Extension.SelectByID("", "EDGE", -0.000802489357837999, -0.0246888176810671, 6.00726028778809E-05,
            true, 0, null, 0);
            swFeat =
            swModel.FeatureManager.InsertCosmeticThread3((int)swCosmeticStandardType_e.swStandardType_StandardHelicoilMetric, "Helicoil
            threads", "M33x2.0", 0.033, (int)swCosmeticEndConditions_e.swEndConditionBlind2Dia, 0.025, "M33x2.0 Helicoil Threads");

            Debug.Print("File = " + swModel.GetPathName());

            swFeat = (Feature)swModel.FirstFeature();

            while ((swFeat != null))
            {
                swSubFeat = (Feature)swFeat.GetFirstSubFeature();
                while ((swSubFeat != null))
                {
                    sFeatType = swSubFeat.GetTypeName();

                    switch (sFeatType)
                    {
                        case "CosmeticThread":
                            Debug.Print(" " + swSubFeat.Name + " [" + sFeatType + "]);

                            swCosThread = (CosmeticThreadFeatureData)swSubFeat.GetDefinition();

                            Debug.Print(" ApplyThread = " + swCosThread.ApplyThread);
                            Debug.Print(" BlindDepth = " + swCosThread.BlindDepth * 1000.0 + " mm");
                            Debug.Print(" Diameter = " + swCosThread.Diameter * 1000.0 + " mm");
                            Debug.Print(" DiameterType = " + swCosThread.DiameterType);
                            Debug.Print(" ThreadCallout = " + swCosThread.ThreadCallout);
                            Debug.Print(" ConfigurationOption as defined in swCosmeticConfigOptions_e = " +
                            swCosThread.ConfigurationOption);
                            Debug.Print(" EndCondition as defined in swCosmeticEndConditions_e = " +
                            swCosThread.EndCondition);
                            Debug.Print(" Size = " + swCosThread.Size);
                            Debug.Print(" Standard as defined in swCosmeticStandardType_e = " + swCosThread.Standard);
                            Debug.Print(" StandardType = " + swCosThread.StandardType);

                            Debug.Print("");

                            break;
                    }

                    swSubFeat = (Feature)swSubFeat.GetNextSubFeature();
                }
            }
        }
    }
}
```

```
        swFeat = (Feature)swFeat.GetNextFeature();  
    }  
}  
public SldWorks swApp;  
}
```

## Traverse All Cosmetic Threads Example (VBA)

This example shows how to traverse all cosmetic threads in a part and extract their data.

**NOTE:** In a part or assembly, a cosmetic thread is a subfeature of a hole or cut extrusion. Thus, you can traverse all of the cosmetic threads in a model using the IFeature traversal methods.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\holecube.sldprt.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a Helicoil Metric standard cosmetic thread.
' 2. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----

```

Option Explicit

Sub main()

```

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swFeat As SldWorks.Feature
    Dim swSubFeat As SldWorks.Feature
    Dim sFeatType As String
    Dim swCosThread As SldWorks.CosmeticThreadFeatureData
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    bRet = swModel.Extension.SelectByID("", "EDGE", -8.02489357837999E-04, -2.46888176810671E-02, 6.00726028778809E-05, True, 0,
    Nothing, 0)
    Set swFeat = swModel.FeatureManager.InsertCosmeticThread3(swStandardType_StandardHelicoilMetric, "Helicoil threads", "M33x2.0",
    0.033, swEndConditionBlind2Dia, 0.025, "M33x2.0 Helicoil Threads")
    Debug.Print "File = " & swModel.GetPathName
    Set swFeat = swModel.FirstFeature
    Do While Not swFeat Is Nothing
        Set swSubFeat = swFeat.GetFirstSubFeature
        Do While Not swSubFeat Is Nothing
            sFeatType = swSubFeat.GetTypeName
            Select Case sFeatType
                Case "CosmeticThread"
                    Debug.Print " " & swSubFeat.Name & " [" & sFeatType & "]"
                    Set swCosThread = swSubFeat.GetDefinition
                    Debug.Print "    ApplyThread      = " & swCosThread.ApplyThread
                    Debug.Print "    BlindDepth     = " & swCosThread.BlindDepth * 1000# & " mm"
                    Debug.Print "    Diameter       = " & swCosThread.Diameter * 1000# & " mm"
                    Debug.Print "    DiameterType   = " & swCosThread.DiameterType
                    Debug.Print "    ThreadCallout  = " & swCosThread.ThreadCallout
                    Debug.Print "    ConfigurationOption as defined in swCosmeticConfigOptions_e = " &
                    swCosThread.ConfigurationOption
                    Debug.Print "    EndCondition as defined in swCosmeticEndConditions_e = " & swCosThread.EndCondition
                    Debug.Print "    Size = " & swCosThread.Size
                    Debug.Print "    Standard as defined in swCosmeticStandardType_e = " & swCosThread.Standard
                    Debug.Print "    StandardType = " & swCosThread.StandardType

                    Debug.Print ""
                End Select
            Set swSubFeat = swSubFeat.GetNextSubFeature
        Loop
        Set swFeat = swFeat.GetNextFeature
    Loop
End Sub

```

## Traverse All Cosmetic Threads Example (VB.NET)

This example shows how to traverse all cosmetic threads in a part and extract their data.

**NOTE:** In a part or assembly, a cosmetic thread is a subfeature of a hole or cut extrusion. Thus, you can traverse all of the cosmetic threads in a model using the IFeature traversal methods.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\holecube.sldprt.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a Helicoil Metric standard cosmetic thread.
' 2. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub main()

        Dim swModel As ModelDoc2
        Dim swFeat As Feature
        Dim swSubFeat As Feature
        Dim sFeatType As String
        Dim swCosThread As CosmeticThreadFeatureData
        Dim bRet As Boolean

        swModel = swApp.ActiveDoc

        bRet = swModel.Extension.SelectByID2("", "EDGE", -0.000802489357837999, -0.0246888176810671, 0.0000600726028778809, True,
0, Nothing, 0)
        swFeat = swModel.FeatureManager.InsertCosmeticThread3(swCosmeticStandardType_e.swStandardType_StandardHelicoilMetric,
"Helicoil threads", "M33x2.0", 0.033, swCosmeticEndConditions_e.swEndConditionBlind2Dia, 0.025, "M33x2.0 Helicoil Threads")

        Debug.Print("File = " & swModel.GetPathName)

        swFeat = swModel.FirstFeature

        Do While Not swFeat Is Nothing
            swSubFeat = swFeat.GetFirstSubFeature
            Do While Not swSubFeat Is Nothing
                sFeatType = swSubFeat.GetTypeName

                Select Case sFeatType

                    Case "CosmeticThread"
                        Debug.Print(" " & swSubFeat.Name & " [" & sFeatType & "]")

                        swCosThread = swSubFeat.GetDefinition

                        Debug.Print(" ApplyThread = " & swCosThread.ApplyThread)
                        Debug.Print(" BlindDepth = " & swCosThread.BlindDepth * 1000.0# & " mm")
                        Debug.Print(" Diameter = " & swCosThread.Diameter * 1000.0# & " mm")
                        Debug.Print(" DiameterType = " & swCosThread.DiameterType)
                        Debug.Print(" ThreadCallout = " & swCosThread.ThreadCallout)
                        Debug.Print(" ConfigurationOption as defined in swCosmeticConfigOptions_e = " &
swCosThread.ConfigurationOption)
                        Debug.Print(" EndCondition as defined in swCosmeticEndConditions_e = " & swCosThread.EndCondition)
                        Debug.Print(" Size = " & swCosThread.Size)
                        Debug.Print(" Standard as defined in swCosmeticStandardType_e = " & swCosThread.Standard)
                        Debug.Print(" StandardType = " & swCosThread.StandardType)

                        Debug.Print("")

                End Select

                swSubFeat = swSubFeat.GetNextSubFeature

            Loop

            swFeat = swFeat.GetNextFeature

        Loop

    End Sub

Public swApp As SldWorks

End Class

```

### Traverse All Open Documents Example (C++ COM)

This example shows how to traverse all open documents.

```
bool CFileReferenceTests::isFileOpen(CString fileSpec)
{

    bool retval = FALSE;
    HRESULT hres = S_FALSE;

    if (fileSpec.IsEmpty())
        return retval;

    LPENUMDOCUMENTS pDocEnum = NULL;
    hres = TheApplication->m_pSldWorks->EnumDocuments2(&pDocEnum);

    // No documents open
    if (pDocEnum == NULL)
        return retval;

    int docNum = 0;
    LPMODELDOC pModelDoc = NULL;
    hres = pDocEnum->Next(1, &pModelDoc, NULL);
    while (S_OK == hres)
    {
        BSTR bdocName;
        hres = pModelDoc->GetPathName(&bdocName);
        CString docName(bdocName);

        // File name matches
        if (fileSpec.CompareNoCase(docName) == 0)
            retval = TRUE;

        pModelDoc->Release();
        pModelDoc = NULL;
        hres = pDocEnum->Next(1, &pModelDoc, NULL);
    }

    if (pDocEnum != NULL) pDocEnum->Release();

    return retval;
}
```

## Traverse Annotations Example (C#)

This example shows how to get display dimension annotations.

```
//-----
// Preconditions:
// 1. Verify that the specified part document exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document and selects
//    a sketch containing multiple dimensions.
// 2. Iterates the display dimensions and gets
//    each display dimension annotation and its position.
// 3. Moves each display dimension annotation 100mm to
//    the right.
// 4. Examine the graphics area and Immediate window.
//
// NOTE: Because the part document is used elsewhere, do not
// save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace TraverseAnnotationsCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Annotation swAnnotation = default(Annotation);
            double[] annotationPosition = null;
            Feature swFeature = default(Feature);
            DisplayDimension swDispDim = default(DisplayDimension);
            string fileName = null;
            int errors = 0;
            int warnings = 0;
            bool status = false;

            //Open part document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\tolanalyst\\offset\\top_plate.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);

            //Get and edit sketch with dimensions
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swModel.EditSketch();

            //Get the first display dimension
            swDispDim = (DisplayDimension)swFeature.GetFirstDisplayDimension();

            //Iterate through all of the display dimension
            //annotations in the sketch
            while ((swDispDim != null))
            {
                Debug.Print("Display dimension annotation name:");
                //Get the display dimension annotation
                swAnnotation = (Annotation)swDispDim.GetAnnotation();
                Debug.Print(" " + swAnnotation.GetName());
                //Get the position of the display dimension annotation
                annotationPosition = (double[])swAnnotation.GetPosition();
                if ((annotationPosition != null))
                {
                    //Move the display dimension annotation 100mm to the right
                    swAnnotation.SetPosition2(annotationPosition[0] + 0.1, annotationPosition[1], annotationPosition[2]);
                }
                //Get the next display dimension
                swDispDim = (DisplayDimension)swFeature.GetNextDisplayDimension(swDispDim);
            }
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
```

## Traverse Annotations Example (VBA)

This example shows how to get display dimension annotations.

```

'-----
' Preconditions:
' 1. Verify that the specified part document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document and selects
'    a sketch containing multiple dimensions.
' 2. Iterates the display dimensions and gets
'    each display dimension annotation and its position.
' 3. Moves each display dimension annotation 100mm to
'    the right.
' 4. Examine the graphics area and Immediate window.
'
' NOTE: Because the part document is used elsewhere, do not
' save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swAnnotation As SldWorks.Annotation
Dim annotationPosition As Variant
Dim swFeature As SldWorks.Feature
Dim swDispDim As SldWorks.DisplayDimension
Dim fileName As String
Dim errors As Long
Dim warnings As Long
Dim status As Boolean

Sub main()

    Set swApp = CreateObject("SldWorks.Application")

    'Open part document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\tolanalyst\offset\top_plate.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Get and edit sketch with dimensions
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelMgr = swModel.SelectionManager
    Set swFeature = swSelMgr.GetSelectedObject6(1, -1)
    swModel.EditSketch

    'Get the first display dimension
    Set swDispDim = swFeature.GetFirstDisplayDimension

    'Iterate through all of the display dimension
    'annotations in the sketch
    Do While Not swDispDim Is Nothing
        Debug.Print "Display dimension annotation name:"
        'Get the display dimension annotation
        Set swAnnotation = swDispDim.GetAnnotation
        Debug.Print " " & swAnnotation.GetName
        'Get the position of the display dimension annotation
        annotationPosition = swAnnotation.GetPosition
        If Not IsNull(annotationPosition) Then
            'Move the display dimension annotation 100mm to the right
            swAnnotation.SetPosition2 annotationPosition(0) + 0.1, annotationPosition(1), annotationPosition(2)
        End If
        'Get the next display dimension
        Set swDispDim = swFeature.GetNextDisplayDimension(swDispDim)
    Loop

End Sub
    
```

## Traverse Annotations Example (VB.NET)

This example shows how to get display dimension annotations.

```

-----
Preconditions:
1. Verify that the specified part document exists.
2. Open the Immediate window.
Postconditions:
1. Opens the specified part document and selects
   a sketch containing multiple dimensions.
2. Iterates the display dimensions and gets
   each display dimension annotation and its position.
3. Moves each display dimension annotation 100mm to
   the right.
4. Examine the graphics area and Immediate window.
NOTE: Because the part document is used elsewhere, do not
save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swAnnotation As Annotation
        Dim annotationPosition() As Double
        Dim swFeature As Feature
        Dim swDispDim As DisplayDimension
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer
        Dim status As Boolean

        'Open part document
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\tolanalyst\offset\top_plate.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        'Get and edit sketch with dimensions
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        swSelMgr = swModel.SelectionManager
        swFeature = swSelMgr.GetSelectedObject6(1, -1)
        swModel.EditSketch()

        'Get the first display dimension
        swDispDim = swFeature.GetFirstDisplayDimension

        'Iterate through all of the display dimension
        'annotations in the sketch
        Do While Not swDispDim Is Nothing
            Debug.Print("Display dimension annotation name:")
            'Get the display dimension annotation
            swAnnotation = swDispDim.GetAnnotation
            Debug.Print(" " & swAnnotation.GetName)
            'Get the position of the display dimension annotation
            annotationPosition = swAnnotation.GetPosition
            If Not annotationPosition Is Nothing Then
                'Move the display dimension annotation 100mm to the right
                swAnnotation.SetPosition2(annotationPosition(0) + 0.1, annotationPosition(1), annotationPosition(2))
            End If
            'Get the next display dimension
            swDispDim = swFeature.GetNextDisplayDimension(swDispDim)
        Loop

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
    
```

## Traverse Assembly and Hide All Sketches Example (VBA)

This example shows how to traverse an assembly and hide all sketches.

```
'-----
' Preconditions:
' 1. Open an assembly document.
' 2. In the FeatureManager design tree, show some sketches by
'    expanding a component, expanding one or more features
'    in that component, right-clicking each sketch in each of
'    the expanded features, and clicking Show in each
'    context-sensitive menu.
' 3. Examine the graphics area and locate the shown sketches.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly document.
' 2. Hides all shown sketches in the assembly.
' 3. Examine the Immediate window and the graphics area.
'-----
Option Explicit

Sub BlankSketchFeature(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swFeat As SldWorks.Feature)
    Dim bRet As Boolean
    If "ProfileFeature" = swFeat.GetTypeName Then
        bRet = swFeat.Select2(False, 0): Debug.Assert bRet
        swModel.BlankSketch
    End If
End Sub

Sub TraverseFeatureFeatures(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swFeat As SldWorks.Feature, nLevel As Long)
    Dim swSubFeat As SldWorks.Feature
    Dim swSubSubFeat As SldWorks.Feature
    Dim swSubSubSubFeat As SldWorks.Feature
    Dim sPadStr As String
    Dim i As Long

    For i = 0 To nLevel
        sPadStr = sPadStr + "  "
    Next i

    Dim bRet As Boolean

    If "Annotations" <> swFeat.Name Then
        bRet = swFeat.Select2(True, 0): Debug.Assert bRet
    End If

    While Not swFeat Is Nothing
        Debug.Print sPadStr + swFeat.Name + " [" + swFeat.GetTypeName + "]"
        BlankSketchFeature swApp, swModel, swFeat
        Set swSubFeat = swFeat.GetFirstSubFeature
        While Not swSubFeat Is Nothing
            Debug.Print sPadStr + "    " + swSubFeat.Name + " [" + swSubFeat.GetTypeName + "]"
            BlankSketchFeature swApp, swModel, swSubFeat
            Set swSubSubFeat = swSubFeat.GetFirstSubFeature
            While Not swSubSubFeat Is Nothing
                Debug.Print sPadStr + "        " + swSubSubFeat.Name + " [" + swSubSubFeat.GetTypeName + "]"
                BlankSketchFeature swApp, swModel, swSubSubFeat
                Set swSubSubSubFeat = swSubSubFeat.GetFirstSubFeature
                While Not swSubSubSubFeat Is Nothing
                    Debug.Print sPadStr + "            " + swSubSubSubFeat.Name + " [" + swSubSubSubFeat.GetTypeName + "]"
                    BlankSketchFeature swApp, swModel, swSubSubSubFeat
                    Set swSubSubSubFeat = swSubSubSubFeat.GetNextSubFeature()
                Wend
                Set swSubSubSubFeat = swSubSubSubFeat.GetNextSubFeature()
            Wend
            Set swSubSubFeat = swSubSubFeat.GetNextSubFeature()
        Wend
        Set swSubFeat = swSubFeat.GetNextSubFeature()
    Wend
    Set swFeat = swFeat.GetNextFeature
Wend
End Sub

Sub TraverseComponentFeatures(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swComp As SldWorks.Component2, nLevel As Long)
    Dim swFeat As SldWorks.Feature

    Set swFeat = swComp.FirstFeature
    TraverseFeatureFeatures swApp, swModel, swFeat, nLevel
End Sub

Sub TraverseComponent(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swComp As SldWorks.Component2, nLevel As Long)
    Dim vChildComp As Variant
    Dim swChildComp As SldWorks.Component2
    Dim swCompConfig As SldWorks.Configuration
    Dim sPadStr As String
    Dim i As Long

    For i = 0 To nLevel - 1
        sPadStr = sPadStr + "  "
    Next i

    vChildComp = swComp.GetChildren
    For i = 0 To UBound(vChildComp)
        Set swChildComp = vChildComp(i)
        Debug.Print sPadStr & "+" & swChildComp.Name2 & " <" & swChildComp.ReferencedConfiguration & ">"
        TraverseComponentFeatures swApp, swModel, swChildComp, nLevel
        TraverseComponent swApp, swModel, swChildComp, nLevel + 1
    Next i
End Sub
```

```

        Next i
    End Sub

    Sub TraverseModelFeatures(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, nLevel As Long)
        Dim swFeat As SldWorks.Feature

        Set swFeat = swModel.FirstFeature
        TraverseFeatureFeatures swApp, swModel, swFeat, nLevel
    End Sub

    Sub main()
        Dim swApp As SldWorks.SldWorks
        Dim swModel As SldWorks.ModelDoc2
        Dim swConf As SldWorks.Configuration
        Dim swRootComp As SldWorks.Component2
        Dim nStart As Long
        Dim bRet As Boolean

        Set swApp = Application.SldWorks
        Set swModel = swApp.ActiveDoc
        Set swConf = swModel.GetActiveConfiguration
        Set swRootComp = swConf.GetRootComponent3(True)

        nStart = Timer

        Debug.Print "File = " & swModel.GetPathName

        ' Traverse the assembly and hide all shown sketches
        TraverseModelFeatures swApp, swModel, 1
        TraverseComponent swApp, swModel, swRootComp, 1

        Debug.Print ""
        Debug.Print "Time to traverse the assembly = " & Timer - nStart & " seconds"

    End Sub

```

## Traversing Assembly at Component and Feature Levels Example (VBA)

This example shows how to traverse an assembly at the component and feature levels.

```
'-----
' Preconditions:
' 1. Verify that the specified assembly document to open
'    exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Specified assembly document opens.
' 2. Traverses the components and features in the assembly.
' 3. Examine the Immediate window.
'-----
```

Option Explicit

Sub TraverseFeatureFeatures(swFeat As SldWorks.Feature, nLevel As Long)

```
    Dim swSubFeat As SldWorks.Feature
    Dim swSubSubFeat As SldWorks.Feature
    Dim swSubSubSubFeat As SldWorks.Feature
    Dim sPadStr As String
    Dim i As Long

    For i = 0 To nLevel
        sPadStr = sPadStr + "  "
    Next i

    While Not swFeat Is Nothing
        Debug.Print sPadStr + swFeat.Name + " [" + swFeat.GetTypeName + "]"
        Set swSubFeat = swFeat.GetFirstSubFeature
        While Not swSubFeat Is Nothing
            Debug.Print sPadStr + "    " + swSubFeat.Name + " [" + swSubFeat.GetTypeName + "]"
            Set swSubSubFeat = swSubFeat.GetFirstSubFeature
            While Not swSubSubFeat Is Nothing
                Debug.Print sPadStr + "      " + swSubSubFeat.Name + " [" + swSubSubFeat.GetTypeName + "]"
                Set swSubSubSubFeat = swSubSubFeat.GetFirstSubFeature
                While Not swSubSubSubFeat Is Nothing
                    Debug.Print sPadStr + "        " + swSubSubSubFeat.Name + " [" + swSubSubSubFeat.GetTypeName + "]"
                    Set swSubSubSubFeat = swSubSubSubFeat.GetNextSubFeature()
                Wend
                Set swSubSubSubFeat = swSubSubSubFeat.GetNextSubFeature()
            Wend
            Set swSubSubFeat = swSubSubFeat.GetNextSubFeature()
        Wend
        Set swSubFeat = swSubFeat.GetNextSubFeature()
    Wend
    Set swFeat = swFeat.GetNextFeature
Wend
```

End Sub

Sub TraverseComponentFeatures(swComp As SldWorks.Component2, nLevel As Long)

```
    Dim swFeat As SldWorks.Feature

    Set swFeat = swComp.FirstFeature
    TraverseFeatureFeatures swFeat, nLevel
```

End Sub

Sub TraverseComponent(swComp As SldWorks.Component2, nLevel As Long)

```
    Dim vChildComp As Variant
    Dim swChildComp As SldWorks.Component2
    Dim swCompConfig As SldWorks.Configuration
    Dim sPadStr As String
    Dim i As Long

    For i = 0 To nLevel - 1
        sPadStr = sPadStr + "  "
    Next i

    vChildComp = swComp.GetChildren
    For i = 0 To UBound(vChildComp)
        Set swChildComp = vChildComp(i)
        Debug.Print sPadStr & "+" & swChildComp.Name2 & " <" & swChildComp.ReferencedConfiguration & ">"
        TraverseComponentFeatures swChildComp, nLevel
        TraverseComponent swChildComp, nLevel + 1
    Next i
```

End Sub

Sub TraverseModelFeatures(swModel As SldWorks.ModelDoc2, nLevel As Long)

```
    Dim swFeat As SldWorks.Feature

    Set swFeat = swModel.FirstFeature
    TraverseFeatureFeatures swFeat, nLevel
```

End Sub

Sub main()

```
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swAssy As SldWorks.AssemblyDoc
    Dim swConf As SldWorks.Configuration
    Dim swRootComp As SldWorks.Component2
```

```

Dim nStart As Long
Dim bRet As Boolean
Dim fileName As String
Dim errors As Long
Dim warnings As Long

Set swApp = CreateObject("SldWorks.Application")

' Open assembly
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\smartcomponents\stepped_shaft.sldasm"
Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

Set swConf = swModel.GetActiveConfiguration
Set swRootComp = swConf.GetRootComponent3(True)

nStart = Timer

Debug.Print "File = " & swModel.GetPathName

' Traverse components and features
TraverseModelFeatures swModel, 1
TraverseComponent swRootComp, 1

Debug.Print ""
Debug.Print "Time = " & Timer - nStart & " seconds"

End Sub

```

## Traverse Assembly at Component and Feature Levels Using Recursion Example (C#)

This example shows how to traverse an assembly at the component and feature levels using recursion.

```
//-----
// Preconditions:
// 1. Open an assembly document containing nested subassemblies.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Traverses the assembly.
// 2. Examine the Immediate Window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace RecursiveTraverseAssemblyCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void TraverseFeatureFeatures(Feature swFeat, long nLevel)
        {
            Feature swSubFeat;
            Feature swSubSubFeat;
            Feature swSubSubSubFeat;
            string sPadStr = " ";
            long i = 0;

            for (i = 0; i <= nLevel; i++)
            {
                sPadStr = sPadStr + " ";
            }
            while ((swFeat != null))
            {
                Debug.Print(sPadStr + swFeat.Name + " [" + swFeat.GetTypeName2() + "]");
                swSubFeat = (Feature)swFeat.GetFirstSubFeature();

                while ((swSubFeat != null))
                {
                    Debug.Print(sPadStr + " " + swSubFeat.Name + " [" + swSubFeat.GetTypeName() + "]");
                    swSubSubFeat = (Feature)swSubFeat.GetFirstSubFeature();

                    while ((swSubSubFeat != null))
                    {
                        Debug.Print(sPadStr + " " + swSubSubFeat.Name + " [" + swSubSubFeat.GetTypeName() + "]");
                        swSubSubSubFeat = (Feature)swSubSubFeat.GetFirstSubFeature();

                        while ((swSubSubSubFeat != null))
                        {
                            Debug.Print(sPadStr + " " + swSubSubSubFeat.Name + " [" + swSubSubSubFeat.GetTypeName() + "]");
                            swSubSubSubFeat = (Feature)swSubSubSubFeat.GetNextSubFeature();
                        }

                        swSubSubSubFeat = (Feature)swSubSubSubFeat.GetNextSubFeature();
                    }

                    swSubSubFeat = (Feature)swSubSubFeat.GetNextSubFeature();
                }

                swSubFeat = (Feature)swSubFeat.GetNextSubFeature();
            }

            swFeat = (Feature)swFeat.GetNextFeature();
        }

        public void TraverseComponentFeatures(Component2 swComp, long nLevel)
        {
            Feature swFeat;

            swFeat = (Feature)swComp.FirstFeature();
            TraverseFeatureFeatures(swFeat, nLevel);
        }

        public void TraverseComponent(Component2 swComp, long nLevel)
        {
            object[] vChildComp;
            Component2 swChildComp;
            string sPadStr = " ";
            long i = 0;

            for (i = 0; i <= nLevel - 1; i++)
            {
                sPadStr = sPadStr + " ";
            }

            vChildComp = (object[])swComp.GetChildren();
            for (i = 0; i < vChildComp.Length; i++)
            {
                swChildComp = (Component2)vChildComp[i];
                Debug.Print(sPadStr + "+" + swChildComp.Name2 + " <" + swChildComp.ReferencedConfiguration + ">");

                TraverseComponentFeatures(swChildComp, nLevel);
                TraverseComponent(swChildComp, nLevel + 1);
            }
        }

        public void TraverseModelFeatures(ModelDoc2 swModel, long nLevel)
        {
            Feature swFeat;

            swFeat = (Feature)swModel.FirstFeature();
            TraverseFeatureFeatures(swFeat, nLevel);
        }
    }
}
```

```

public void Main()
{
    ModelDoc2 swModel;
    ConfigurationManager swConfMgr;
    Configuration swConf;
    Component2 swRootComp;

    swModel = (ModelDoc2)swApp.ActiveDoc;
    swConfMgr = (ConfigurationManager)swModel.ConfigurationManager;
    swConf = (Configuration)swConfMgr.ActiveConfiguration;
    swRootComp = (Component2)swConf.GetRootComponent();

    System.Diagnostics.Stopwatch myStopwatch = new Stopwatch();
    myStopwatch.Start();

    Debug.Print("File = " + swModel.GetPathName());

    TraverseModelFeatures(swModel, 1);

    if (swModel.GetType() == (int)swDocumentTypes_e.swDocASSEMBLY)
    {
        TraverseComponent(swRootComp, 1);
    }

    myStopwatch.Stop();
    TimeSpan myTimespan = myStopwatch.Elapsed;
    Debug.Print("Time = " + myTimespan.TotalSeconds + " sec");
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

## Traverse Assembly at Component and Feature Levels Using Recursion Example (VBA)

This example shows how to traverse an assembly at the component and feature levels using recursion.

```
'-----
' Preconditions:
' 1. Open an assembly document containing nested subassemblies.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Traverses the assembly.
' 2. Examine the Immediate window.
'-----
```

Option Explicit

Sub TraverseFeatureFeatures(swFeat As SldWorks.Feature, nLevel As Long)

```
Dim swSubFeat As SldWorks.Feature
Dim swSubSubFeat As SldWorks.Feature
Dim swSubSubSubFeat As SldWorks.Feature
Dim sPadStr As String
Dim i As Long

For i = 0 To nLevel
    sPadStr = sPadStr + " "
Next i

While Not swFeat Is Nothing
    Debug.Print sPadStr + swFeat.Name + " [" + swFeat.GetTypeName + "]"

    Set swSubFeat = swFeat.GetFirstSubFeature
    While Not swSubFeat Is Nothing
        Debug.Print sPadStr + " " + swSubFeat.Name + " [" + swSubFeat.GetTypeName + "]"

        Set swSubSubFeat = swSubFeat.GetFirstSubFeature
        While Not swSubSubFeat Is Nothing
            Debug.Print sPadStr + " " + swSubSubFeat.Name + " [" + swSubSubFeat.GetTypeName + "]"

            Set swSubSubSubFeat = swSubSubFeat.GetFirstSubFeature
            While Not swSubSubSubFeat Is Nothing
                Debug.Print sPadStr + " " + swSubSubSubFeat.Name + " [" + swSubSubSubFeat.GetTypeName + "]"

                Set swSubSubSubFeat = swSubSubSubFeat.GetNextSubFeature()
            Wend

            Set swSubSubSubFeat = swSubSubSubFeat.GetNextSubFeature()
        Wend

        Set swSubSubFeat = swSubSubFeat.GetNextSubFeature()
    Wend

    Set swSubFeat = swSubFeat.GetNextSubFeature()
Wend

Set swFeat = swFeat.GetNextFeature

Wend

End Sub
```

Sub TraverseComponentFeatures(swComp As SldWorks.Component2, nLevel As Long)

```
Dim swFeat As SldWorks.Feature

Set swFeat = swComp.FirstFeature
TraverseFeatureFeatures swFeat, nLevel
```

End Sub

Sub TraverseComponent(swComp As SldWorks.Component2, nLevel As Long)

```
Dim vChildComp As Variant
Dim swChildComp As SldWorks.Component2
Dim sPadStr As String
Dim i As Long

For i = 0 To nLevel - 1
    sPadStr = sPadStr + " "
Next i

vChildComp = swComp.GetChildren
For i = 0 To UBound(vChildComp)
    Set swChildComp = vChildComp(i)

    Debug.Print sPadStr + "+" & swChildComp.Name2 & " <" & swChildComp.ReferencedConfiguration & ">"

    TraverseComponentFeatures swChildComp, nLevel
    TraverseComponent swChildComp, nLevel + 1

Next i
```

End Sub

Sub TraverseModelFeatures(swModel As SldWorks.ModelDoc2, nLevel As Long)

```

Dim swFeat As SldWorks.Feature

Set swFeat = swModel.FirstFeature
TraverseFeatureFeatures swFeat, nLevel

End Sub

Sub main()

Dim swApp                As SldWorks.SldWorks
Dim swModel              As SldWorks.ModelDoc2
Dim swConfMgr            As SldWorks.ConfigurationManager
Dim swConf               As SldWorks.Configuration
Dim swRootComp           As SldWorks.Component2
Dim StartTime            As Double
Dim FinishTime           As Double
Dim TotalTime            As Double

Set swApp = CreateObject("SldWorks.Application")
Set swModel = swApp.ActiveDoc
Set swConfMgr = swModel.ConfigurationManager
Set swConf = swConfMgr.ActiveConfiguration
Set swRootComp = swConf.GetRootComponent3(True)

StartTime = Timer ' Start time

Debug.Print "File = " & swModel.GetPathName

TraverseModelFeatures swModel, 1

If swModel.GetType = SwConst.swDocASSEMBLY Then
    TraverseComponent swRootComp, 1
End If

FinishTime = Timer ' End time
TotalTime = FinishTime - StartTime ' Elapsed time
Debug.Print ("Time = " & TotalTime & " sec")

End Sub

```

## Traverse Assembly at Component and Feature Levels Using Recursion Example (VB.NET)

This example shows how to traverse an assembly at the component and feature levels using recursion.

```

-----
Preconditions:
1. Open an assembly document containing nested subassemblies.
2. Open the Immediate window.
Postconditions:
1. Traverses the assembly.
2. Examine the Immediate window.
-----
    
```

Option Explicit On

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
Imports System.Diagnostics.Stopwatch
    
```

Partial Class SolidWorksMacro

```

Sub TraverseFeatureFeatures(ByVal swFeat As Feature, ByVal nLevel As Long)
    Dim swSubFeat As Feature
    Dim swSubSubFeat As Feature
    Dim swSubSubSubFeat As Feature
    Dim sPadStr As String = " "
    Dim i As Long

    For i = 0 To nLevel
        sPadStr = sPadStr + " "
    Next i

    While Not swFeat Is Nothing
        Debug.Print(sPadStr + swFeat.Name + " [" + swFeat.GetTypeName2 + "]")
        swSubFeat = swFeat.GetFirstSubFeature

        While Not swSubFeat Is Nothing
            Debug.Print(sPadStr + " " + swSubFeat.Name + " [" + swSubFeat.GetTypeName + "]")
            swSubSubFeat = swSubFeat.GetFirstSubFeature

            While Not swSubSubFeat Is Nothing
                Debug.Print(sPadStr + " " + swSubSubFeat.Name + " [" + swSubSubFeat.GetTypeName + "]")
                swSubSubSubFeat = swSubSubFeat.GetFirstSubFeature

                While Not swSubSubSubFeat Is Nothing
                    Debug.Print(sPadStr + " " + swSubSubSubFeat.Name + " [" + swSubSubSubFeat.GetTypeName + "]")
                    swSubSubSubSubFeat = swSubSubSubFeat.GetNextSubFeature()

                End While

                swSubSubSubFeat = swSubSubSubFeat.GetNextSubFeature()

            End While

            swSubSubFeat = swSubSubFeat.GetNextSubFeature()

        End While

        swSubFeat = swSubFeat.GetNextSubFeature()

    End While

    swFeat = swFeat.GetNextFeature

End While

End Sub

Sub TraverseComponentFeatures(ByVal swComp As Component2, ByVal nLevel As Long)
    Dim swFeat As Feature

    swFeat = swComp.FirstFeature
    TraverseFeatureFeatures(swFeat, nLevel)

End Sub

Sub TraverseComponent(ByVal swComp As Component2, ByVal nLevel As Long)
    Dim vChildComp As Object
    Dim swChildComp As Component2
    Dim sPadStr As String = " "
    Dim i As Long

    For i = 0 To nLevel - 1
        sPadStr = sPadStr + " "
    Next i

    vChildComp = swComp.GetChildren
    For i = 0 To UBound(vChildComp)
        swChildComp = vChildComp(i)

        Debug.Print(sPadStr + "+" & swChildComp.Name2 & " <" & swChildComp.ReferencedConfiguration & ">")

        TraverseComponentFeatures(swChildComp, nLevel)
        TraverseComponent(swChildComp, nLevel + 1)

    Next i

End Sub

Sub TraverseModelFeatures(ByVal swModel As ModelDoc2, ByVal nLevel As Long)
    Dim swFeat As Feature

    swFeat = swModel.FirstFeature
    TraverseFeatureFeatures(swFeat, nLevel)

End Sub

Public Sub main()

    Dim swApp As SldWorks
    Dim swModel As ModelDoc2
    Dim swConfMgr As ConfigurationManager
    Dim swConf As Configuration
    Dim swRootComp As Component2
    
```

```

Dim StartTime As Double
Dim FinishTime As Double
Dim TotalTime As Double

swApp = CreateObject("SldWorks.Application")
swModel = swApp.ActiveDoc
swConfMgr = swModel.ConfigurationManager
swConf = swConfMgr.ActiveConfiguration
swRootComp = swConf.GetRootComponent3(True)
StartTime = Timer ' Start time
Debug.Print("File = " & swModel.GetPathName)
TraverseModelFeatures(swModel, 1)

If swModel.GetType = swDocumentTypes_e.swDocASSEMBLY Then
    TraverseComponent(swRootComp, 1)

End If

FinishTime = Timer ' End time
TotalTime = FinishTime - StartTime ' Elapsed time
Debug.Print("Time = " & TotalTime & " sec")

```

End Sub

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

## Traverse Assembly at Component Level Example (VBA)

This example shows how to traverse an assembly at the component level.

```

'-----
' Preconditions:
' 1. Verify that the specified assembly document to open
'    exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Specified assembly document opens.
' 2. Traverses the components in the assembly.
' 3. Examine the Immediate window.
'-----
Option Explicit

Sub TraverseComponent _
    (swComp As SldWorks.Component2, nLevel As Long)

    Dim vChildComp As Variant
    Dim swChildComp As SldWorks.Component2
    Dim swCompConfig As SldWorks.Configuration
    Dim sPadStr As String
    Dim i As Long

    For i = 0 To nLevel - 1
        sPadStr = sPadStr + "  "
    Next i

    vChildComp = swComp.GetChildren
    For i = 0 To UBound(vChildComp)
        Set swChildComp = vChildComp(i)
        TraverseComponent swChildComp, nLevel + 1
        Debug.Print sPadStr & swChildComp.Name2 & " <" & swChildComp.ReferencedConfiguration & ">"
    Next i
End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swAssy As SldWorks.AssemblyDoc
    Dim swConf As SldWorks.Configuration
    Dim swRootComp As SldWorks.Component2
    Dim bRet As Boolean
    Dim fileName As String
    Dim errors As Long
    Dim warnings As Long

    Set swApp = CreateObject("SldWorks.Application")

    ' Open assembly
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\smartcomponents\stepped_shaft.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    Set swConf = swModel.GetActiveConfiguration
    Set swRootComp = swConf.GetRootComponent3(True)
    Debug.Print "File = " & swModel.GetPathName

    ' Traverse components
    TraverseComponent swRootComp, 1
End Sub

```

## Traverse Assembly Example (C++ COM)

This example shows how to traverse an assembly using the IComponent2 object. The method IComponent2::IGetChildren returns an array, so this code must be used in an in-process DLL. Otherwise, use the method IComponent2::GetChildren that returns a VARIANT.

```

void GetModelAssembly(ISldWorks* m_pSldWorks)
{
    LPCONFIGURATION pConfiguration = NULL;
    LPCOMPONENT pRootComponent = NULL;

    LPMODELDOC2 pModelDoc = NULL;
    HRESULT hres = S_OK;
    long RecurseLevel = 0;

    hres = m_pSldWorks->get_IActiveDoc2( &pModelDoc );

    // Retrieve model document pointer
    if( S_OK != hres || pModelDoc == NULL )
        return;

    // Get the active configuration and root component
    if ( pModelDoc->IGetActiveConfiguration(&pConfiguration) == S_OK )
    {
        if ( pConfiguration->IGetRootComponent(&pRootComponent) == S_OK )
        {
            CString MyString;
            TraverseChildren(RecurseLevel,&MyString,pRootComponent);

            pRootComponent->Release();

            // Display the structure in a message box
            AfxMessageBox (MyString);
        }
        pConfiguration->Release();
    }
    pModelDoc->Release();
}

////////////////////////////////////
// Function: TraverseChildren
// Arguments: RecurseLevel - Level of recursion
//   MyString - Textural record of assembly
//   pComponent - Component to traverse
// Return: Number of children
// Description: Perform any operations specific to the component, then
//   if the component has children, recursively call
//   this function for each child.
////////////////////////////////////
int TraverseChildren(long RecurseLevel, CString* MyString, LPCOMPONENT pComponent)
{
    LPCOMPONENT* pChildren = NULL;
    int  nChildren;
    int  i;
    BSTR  Name;
    HRESULT  hres = S_OK;
    LPMODELDOC  pModelDoc = NULL;

    // Retrieve the component name
    if(RecurseLevel==0)
    {
        // Special case of top-level components
        hres = m_pSldWorks->get_IActiveDoc( &pModelDoc );
        if( S_OK == hres || pModelDoc != NULL )
            hres = pModelDoc->GetTitle(&Name);
    }
}

```

```

else
{
    // Get the component name
    hres = pComponent->get_Name(&Name);
}

if( S_OK == hres && Name != NULL )
{
    CString tempstr;
    for( i=1; i<=RecurseLevel; i++)
    {
        tempstr += " ";
    }

    CString Tmp(Name);
    tempstr += Tmp;
    tempstr += "\n\n";
    *MyString = *MyString + tempstr;
}

RecurseLevel++;

hres = pComponent->IGetChildrenCount(&nChildren);

// Check if this component has children
if ( S_OK == hres || nChildren > 0 )
{
    pChildren = new LPCOMPONENT [nChildren];

    hres = pComponent->IGetChildren((LPCOMPONENT**)&pChildren);
    if(S_OK == hres)
    {
        // Recursively traverse the children
        for ( i=0; i<nChildren; i++ )
        {
            TraverseChildren(RecurseLevel,MyString,pChildren[i]);
            pChildren[i]->Release();
        }
    }
    delete [] pChildren;
}

RecurseLevel--;
return nChildren;
}

```

## Traverse Bodies (C++/CLI)

This example shows how to traverse bodies.

```
//-----
// BodyTraversal.cpp : main project file.
//
// Preconditions: Model document is open.
//
// Postconditions: None
//-----

#include "stdafx.h"

using namespace System;
// Add the SOLIDWORKS primary interop assemblies to the references
using namespace SOLIDWORKS::Interop::sldworks;
using namespace SOLIDWORKS::Interop::swconst;

int main(array<System::String ^> ^args)
{
    ISldWorks^    swApp;

    swApp = gcnew SldWorksClass;

    if (! swApp) {
        return(0);
    }

    swApp->UserControl = true;
    swApp->Visible = true;

    IModelDoc2^    swModel;
    swModel = swApp->IActiveDoc2;

    if (! swModel) {
        return(0);
    }

    IModelDocExtension^    swModelExtension;
    swModelExtension = swModel->Extension;

    String^        strModelTitle;
    swDocumentTypes_e    nDocumentType;

    strModelTitle = swModel->GetTitle();
    nDocumentType = (swDocumentTypes_e)swModel->GetType();

    if (nDocumentType != swDocumentTypes_e::swDocPART) {
        return(0);
    }

    IPartDoc^    swPart = nullptr;
    swPart = (IPartDoc^)swModel;

    array<Object^>^    aBodies;

    aBodies = safe_cast<array<Object^>^>(swPart->GetBodies2((int)swBodyType_e::swSolidBody, true));

    swModel->ClearSelection2(true);
}
```

```

for each (IBody2^ swBody in aBodies) {
    System::Console::WriteLine("Old name = " + swBody->Name);
    swBody->Name = swBody->Name + "_renamed";
    bool bSelected = swModelExtension->SelectByID2(swBody->Name, "SOLIDBODY", 0.0, 0.0, 0.0, true, 0, nullptr,
    (int)swSelectOption_e::swSelectOptionDefault);
    System::Console::WriteLine("New name = " + swBody->Name);
    IFace2^ swFace = nullptr;
    //swFace = (IFace2^)swBody->GetFirstFace();
    swFace = swBody->IGetFirstFace();
    while (swFace != nullptr) {
        int iNumEdges = swFace->GetEdgeCount();
        //swFace = (IFace2^)swFace->GetNextFace();
        swFace = swFace->IGetNextFace();
    }
}

bool bStatus = swModel->EditRebuild3();

return(0);
}

```

## Traverse Drawing FeatureManager Design Tree Example (VBA)

This example shows how to get a reference to the model underlying the drawing view and then how to traverse the FeatureManager design tree from there.

```

'-----
' Preconditions:
' 1. Open a drawing.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the reference to the model underlying
'    the drawing view.
' 2. Traverses the rest of the FeatureManager
'    design tree.
' 3. Examine the Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swDrawModel As SldWorks.ModelDoc2
    Dim swDraw As SldWorks.DrawingDoc
    Dim swView As SldWorks.View
    Dim sModelName As String
    Dim nDocType As Long
    Dim nErrors As Long
    Dim nWarnings As Long

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel

    ' Drawing sheet
    Set swView = swDraw.GetFirstView

    ' First view on the sheet
    Set swView = swView.GetNextView

    ' Determine if this is a view of a part or assembly
    sModelName = swView.GetReferencedModelName
    sModelName = LCase(sModelName)
    If InStr(sModelName, ".sldprt") Then
        nDocType = swDocPART
    Else
        nDocType = swDocASSEMBLY
    End If

    ' Get reference to underlying model
    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " View = " & swView.Name
    Debug.Print " Model = " & sModelName

    Debug.Print "Continue traversing the FeatureManager design tree"
    Set swView = swView.GetNextView
    Do While Not swView Is Nothing
        Debug.Print " View = " & swView.Name
        Set swView = swView.GetNextView
    Loop

End Sub

```

### Traverse Feature Dimensions Example (VBA)

This example shows how to traverse all of the DisplayDimension objects belonging to the feature, get the underlying Dimension object, and query its value.

```
'-----  
' Preconditions:  
' 1. Open a part.  
' 2. Select a feature.  
'  
' Postconditions:  
' 1. Gets the display dimensions for the selected feature.  
' 2. Click OK to close each message box.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.PartDoc  
Dim Feature As SldWorks.Feature  
Dim theDimen As SldWorks.Dimension  
Dim theDispDimen As SldWorks.DisplayDimension  
Dim thevalue As Double  
Dim i As Long  
  
Sub main()  
    Set swApp = CreateObject("SldWorks.Application")  
    Set Part = swApp.ActiveDoc  
  
    Set Feature = Part.SelectionManager.GetSelectedObject6(1, -1)  
    If (Feature Is Nothing) Then  
        Exit Sub  
    End If  
  
    Set theDispDimen = Feature.GetFirstDisplayDimension  
    i = 0  
    While (Not theDispDimen Is Nothing)  
        i = i + 1  
        Set theDimen = theDispDimen.GetDimension  
        thevalue = theDimen.Value  
        swApp.SendMsgToUser "Dimension Value " + Str(i) + " = " + Str(thevalue)  
        Set theDispDimen = Feature.GetNextDisplayDimension(theDispDimen)  
    Wend  
  
End Sub
```

## Traverse FeatureManager Design Tree Example (VBA)

This example shows how to traverse a FeatureManager design tree using ITreeControlItem.

```
'-----
' Preconditions:
' 1. Open a part or assembly.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Traverses the FeatureManager design
'    tree.
' 2. Examine the Immediate window.
'-----
Option Explicit

Dim traverseLevel As Long

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim myModel As SldWorks.ModelDoc2
    Dim featureMgr As SldWorks.FeatureManager
    Dim rootNode As SldWorks.TreeControlItem

    Set swApp = Application.SldWorks
    Set myModel = swApp.ActiveDoc
    Set featureMgr = myModel.FeatureManager
    Set rootNode = featureMgr.GetFeatureTreeRootItem()
    If Not rootNode Is Nothing Then
        Debug.Print
        traverseLevel = 0
        traverse_node rootNode
    End If

End Sub

Private Sub traverse_node(node As SldWorks.TreeControlItem)
    Dim childNode As SldWorks.TreeControlItem
    Dim featureNode As SldWorks.Feature
    Dim componentNode As SldWorks.Component2
    Dim nodeObjectType As Long
    Dim nodeObject As Object
    Dim restOfString As String
    Dim indent As String
    Dim i As Long
    Dim displayNodeInfo As Boolean
    Dim compName As String
    Dim suppr As Long, supprString As String
    Dim vis As Long, visString As String
    Dim fixed As Boolean, fixedString As String
    Dim componentDoc As Object, docString As String
    Dim refConfigName As String

    displayNodeInfo = False
    nodeObjectType = node.ObjectType
    Set nodeObject = node.Object
    Select Case nodeObjectType
    Case SwConst.swTreeControlItemType_e.swFeatureManagerItem_Feature:
        displayNodeInfo = True
        If Not nodeObject Is Nothing Then
            Set featureNode = nodeObject
            restOfString = "[FEATURE: " & featureNode.Name & "]"
        Else
            restOfString = "[FEATURE: object null?]"
        End If
    Case SwConst.swTreeControlItemType_e.swFeatureManagerItem_Component:
        displayNodeInfo = True
        If Not nodeObject Is Nothing Then
            Set componentNode = nodeObject
            compName = componentNode.Name2
            If (compName = "") Then
                compName = "?"
            End If
            suppr = componentNode.GetSuppression()
            Select Case (suppr)
            Case SwConst.swComponentSuppressionState_e.swComponentFullyResolved
                supprString = "Resolved"
            Case SwConst.swComponentSuppressionState_e.swComponentLightweight
                supprString = "Lightweight"
            Case SwConst.swComponentSuppressionState_e.swComponentSuppressed
                supprString = "Suppressed"
            End Select

            vis = componentNode.Visible

            Select Case (vis)
            Case SwConst.swComponentVisibilityState_e.swComponentHidden
                visString = "Hidden"
            Case SwConst.swComponentVisibilityState_e.swComponentVisible
                visString = "Visible"
            End Select

            fixed = componentNode.IsFixed
        End If
    End Select
End Sub
```

```

        If fixed = 0 Then
            fixedString = "Floating"
        Else
            fixedString = "Fixed"
        End If

        Set componentDoc = componentNode.GetModelDoc

        If componentDoc Is Nothing Then
            docString = "Not loaded"
        Else
            docString = "Loaded"
        End If

        refConfigName = componentNode.ReferencedConfiguration

        If (refConfigName = "") Then
            refConfigName = "?"
        End If

        restOfString = "[COMPONENT: " & compName & " " & docString & " " & supprString & " " & visString & " " & refConfigName & "]"
    Else
        restOfString = "[COMPONENT: object null?]"
    End If
Case Else:
    displayNodeInfo = True
    If Not nodeObject Is Nothing Then
        restOfString = "[object type not handled]"
    Else
        restOfString = "[object null?]"
    End If
End Select

For i = 1 To traverseLevel
    indent = indent & " "
Next i

If (displayNodeInfo) Then
    Debug.Print indent & node.Text & " : " & restOfString
End If

traverseLevel = traverseLevel + 1
Set childNode = node.GetFirstChild()
While Not childNode Is Nothing
    traverse_node childNode
    Set childNode = childNode.GetNext
Wend

traverseLevel = traverseLevel - 1
End Sub

```

### Traverse Features By Reverse Position Example (C#)

This example shows how to traverse backwards through the list of features in the FeatureManager design tree. Features are obtained by their position using IModelDoc2::FeatureByPositionReverse.

```
//-----  
// Preconditions:  
// 1. A part document is open in SOLIDWORKS Design.  
// 2. Open the Immediate window.  
// 3. Run the macro.  
//  
// Postconditions: Examine the Immediate window for  
// the position and names of the features in  
// the FeatureManager design tree in reverse  
// chronological order.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace TraverseFeatureByReverseCSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
        ModelDoc2 Part;  
        Feature theFeature;  
        int featCount;  
        string featName;  
        int i;  
  
        public void Main()  
        {  
            Part = (ModelDoc2)swApp.ActiveDoc;  
  
            featCount = Part.GetFeatureCount();  
            for (i = featCount; i >= 1; i += -1)  
            {  
                theFeature = (Feature)Part.FeatureByPositionReverse(featCount - i);  
                if ((theFeature != null))  
                {  
                    featName = theFeature.Name;  
                    Debug.Print("Feature " + i.ToString() + " is " + featName);  
                }  
            }  
  
            Part = null;  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
  
        public SldWorks swApp;  
    }  
}
```

### Traverse Features By Reverse Position Example (VBA)

This example shows how to traverse backwards through the list of features in the FeatureManager design tree. Features are obtained by their position using IModelDoc2::FeatureByPositionReverse.

```
'-----  
' Preconditions:  
' 1. A part document is open in SOLIDWORKS Design.  
' 2. Open the Immediate window.  
' 3. Run the macro.  
'  
' Postconditions: Examine the Immediate window for  
' the position and names of the features in the  
' FeatureManager design tree in reverse  
' chronological order.  
'-----  
  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim theFeature As SldWorks.Feature  
Dim featCount As Long  
Dim featName As String  
Dim i As Long  
  
Sub main()  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set Part = swApp.ActiveDoc  
  
    featCount = Part.GetFeatureCount  
    For i = featCount To 1 Step -1  
        Set theFeature = Part.FeatureByPositionReverse(featCount - i)  
        If Not theFeature Is Nothing Then  
            featName = theFeature.Name  
            Debug.Print "Feature " + Str(i) + " is " + featName  
        End If  
    Next  
  
    Set Part = Nothing  
  
End Sub
```

Traverse Features By Reverse Position Example (VB.NET)

This example shows how to traverse backwards through the list of features in the FeatureManager design tree. Features are obtained by their position using IModelDoc2::FeatureByPositionReverse.

```

'-----
' Preconditions:
' 1. A part document is open in SOLIDWORKS Design.
' 2. Open the Immediate window.
' 3. Run the macro.
'
' Postconditions: Examine the Immediate window for
' the position and names of the features in the
' FeatureManager design tree in reverse
' chronological order.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Runtime.InteropServices
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim Part As ModelDoc2
    Dim theFeature As Feature
    Dim featCount As Integer
    Dim featName As String
    Dim i As Integer

    Sub Main()

        Part = swApp.ActiveDoc

        featCount = Part.GetFeatureCount
        For i = featCount To 1 Step -1
            theFeature = Part.FeatureByPositionReverse(featCount - i)
            If Not theFeature Is Nothing Then
                featName = theFeature.Name
                Debug.Print("Feature " + Str(i) + " is " + featName)
            End If
        Next

        Part = Nothing

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

## Traverse Hierarchy of Configurations Example (VBA)

This examples shows how to traverse a hierarchy of configurations.

```

'-----
' Preconditions:
' 1. Open a part or assembly.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Traverses the configurations and prints
'    each configuration's information to the
'    Immediate window.
' 2. Examine the Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim vConfigNameArr As Variant
    Dim vConfigName As Variant
    Dim swConfig As SldWorks.Configuration
    Dim swParentConfig As SldWorks.Configuration
    Dim swConfMgr As SldWorks.ConfigurationManager
    Dim vChildConfigArr As Variant
    Dim vChildConfig As Variant
    Dim swChildConfig As SldWorks.Configuration

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swConfMgr = swModel.ConfigurationManager
    Set swConfig = swConfMgr.ActiveConfiguration
    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " Active configuration = " & swConfig.Name
    Debug.Print ""

    ' Always at least one configuration exists
    vConfigNameArr = swModel.GetConfigurationNames
    For Each vConfigName In vConfigNameArr
        Set swConfig = swModel.GetConfigurationByName(vConfigName)
        Debug.Print " Configuration name = " & swConfig.Name
        Debug.Print " Description = " & swConfig.Description
        Debug.Print " Comment = " & swConfig.Comment
        Debug.Print " Use alternate name om BOM? " & swConfig.UseAlternateNameInBOM; ""
        Debug.Print " Alternate name = " & swConfig.AlternateName
        Debug.Print " Show child components in BOM? " & swConfig.ShowChildComponentsInBOM
        Debug.Print " Hide new component models? " & swConfig.HideNewComponentModels
        Debug.Print " Suppress new component models? " & swConfig.SuppressNewComponentModels
        Debug.Print " Suppress new features? " & swConfig.SuppressNewFeatures
        Debug.Print " Stream name = " & swConfig.GetStreamName
        Debug.Print " Is derived? " & swConfig.IsDerived

        ' Process parent
        Set swParentConfig = swConfig.GetParent

        If Not swParentConfig Is Nothing Then
            Debug.Print " Parent = " & swParentConfig.Name
        End If

        ' Process children
        vChildConfigArr = swConfig.GetChildren
        If Not IsEmpty(vChildConfigArr) Then
            For Each vChildConfig In vChildConfigArr
                Set swChildConfig = vChildConfig
                Debug.Print " Child = " & swChildConfig.Name
            Next
        End If
        Debug.Print ""
    Next

End Sub
    
```

### Traverse Subfeatures Example (VBA)

This example shows how to traverse the subfeatures of each feature in a part.

```
'-----
' Preconditions: Open a part that has at least one feature.
'
' Postconditions:
' 1. Gets all of the features and subfeatures.
' 2. Click OK to close each message box.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swPartDoc As SldWorks.ModelDoc2
Dim swFeature As SldWorks.feature
Dim swSubFeature As SldWorks.feature
Dim featureName As String
Dim subFeatureName As String
Dim message As String

Sub main()

    Set swApp = Application.SldWorks
    Set swPartDoc = swApp.ActiveDoc

    ' Get the first feature in part
    Set swFeature = swPartDoc.FirstFeature

    ' While we have a valid feature
    While Not swFeature Is Nothing

        ' Get the name of the feature
        featureName = swFeature.Name
        message = "Feature: " & featureName & Chr(10) & " SubFeatures:"
        Set swSubFeature = swFeature.GetFirstSubFeature

        ' While we have a valid sub-feature
        While Not swSubFeature Is Nothing

            ' Get the name of the sub-feature
            subFeatureName = swSubFeature.Name
            message = message & Chr(10) & "    " & subFeatureName

            Set swSubFeature = swSubFeature.GetNextSubFeature
            ' Continue until the last sub-feature is found
        Wend

        ' Display the sub-features
        swApp.SendMsgToUser message

        ' Get the next feature
        Set swFeature = swFeature.GetNextFeature()

        ' Continue until the last feature is found
    Wend

End Sub
```

## Trim Sketch Entities Example (C#)

This example shows how to trim a sketch to a corner.

```
//-----
// Preconditions: Ensure the specified template exists.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Sketches some lines.
// 3. Examine the sketch, then press
//    F5.
// 4. Selects two lines and trims them
//    to a corner.
// 5. Examine the sketch to verify.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace SketchTrimCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchMgr = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            bool status = false;

            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2018\\templates\\Part.prt", 0, 0, 0);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSketchMgr = (SketchManager)swModel.SketchManager;

            // Create sketch of lines
            status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSketchMgr.InsertSketch(true);
            swModel.ClearSelection2(true);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateLine(-0.055275, 0.03236, 0.0, 0.027405, 0.03236, 0.0);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateLine(0.027405, 0.03236, 0.0, 0.027405, -0.026476, 0.0);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateLine(0.027405, -0.026476, 0.0, -0.055275, -0.026476, 0.0);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateLine(-0.055275, -0.026476, 0.0, -0.055275, -0.070758, 0.0);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateLine(-0.055275, -0.070758, 0.0, 0.027405, -0.070758, 0.0);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateLine(0.027405, -0.070758, 0.0, 0.076642, 0.03236, 0.0);
            swModel.ClearSelection2(true);

            System.Diagnostics.Debugger.Break();
            // Examine the sketch before trimming
            // the selected lines to a corner
            // Press F5
            // Select two lines to trim to a corner

            status = swModelDocExt.SelectByID("Line6", "SKETCHSEGMENT", 0.0391723509933775, -0.0466042594822396, 0, true, 0, null,
            status = swModelDocExt.SelectByID("Line3", "SKETCHSEGMENT", 0.0193539283564118, -0.0264761739915713, 0, true, 0, null,
            status = swSketchMgr.SketchTrim((int)swSketchTrimChoice_e.swSketchTrimCorner, 0, 0, 0);
            swModel.ClearSelection2(true);
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>

    public SldWorks swApp;
}
```

## Trim Sketch Entities Example (VBA)

This example shows how to trim a sketch to a corner.

```
'-----
' Preconditions: Ensure the specified template exists.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Sketches some lines.
' 3. Examine the sketch, then press
'    F5.
' 4. Selects two lines and trims them
'    to a corner.
' 5. Examine the sketch to verify.
'-----

Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchMgr As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2018\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension
    Set swSketchMgr = swModel.SketchManager

    ' Create sketch of lines
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swSketchMgr.InsertSketch True
    swModel.ClearSelection2 True
    Set swSketchSegment = swSketchMgr.CreateLine(-0.055275, 0.03236, 0#, 0.027405, 0.03236, 0#)
    Set swSketchSegment = swSketchMgr.CreateLine(0.027405, 0.03236, 0#, 0.027405, -0.026476, 0#)
    Set swSketchSegment = swSketchMgr.CreateLine(0.027405, -0.026476, 0#, -0.055275, -0.026476, 0#)
    Set swSketchSegment = swSketchMgr.CreateLine(-0.055275, -0.026476, 0#, -0.055275, -0.070758, 0#)
    Set swSketchSegment = swSketchMgr.CreateLine(-0.055275, -0.070758, 0#, 0.027405, -0.070758, 0#)
    Set swSketchSegment = swSketchMgr.CreateLine(0.027405, -0.070758, 0#, 0.076642, 0.03236, 0#)
    swModel.ClearSelection2 True

    Stop
    ' Examine the sketch before trimming
    ' the selected lines to a corner
    ' Press F5

    ' Select two lines to trim to a corner
    status = swModelDocExt.SelectByID2("Line6", "SKETCHSEGMENT", 3.91723509933775E-02, -4.66042594822396E-02, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 1.93539283564118E-02, -2.64761739915713E-02, 0, True, 0, Nothing, 0)
    status = swSketchMgr.SketchTrim(swSketchTrimChoice_e.swSketchTrimCorner, 0, 0, 0)

    swModel.ClearSelection2 True

End Sub
```

## Trim Sketch Entities Example (VB.NET)

This example shows how to trim a sketch to a corner.

```
'-----
' Preconditions: Ensure the specified template exists.
'
```

```
' Postconditions:
' 1. Opens a new part document.
' 2. Sketches some lines.
' 3. Examine the sketch, then press
'    F5.
' 4. Selects two lines and trims them
'    to a corner.
' 5. Examine the sketch to verify.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub Main()
```

```
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchMgr As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim status As Boolean
```

```
        swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2018\templates\Part.prtdot", 0, 0, 0)
        swModelDocExt = swModel.Extension
        swSketchMgr = swModel.SketchManager
```

```
        ' Create sketch of lines
        status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swSketchMgr.InsertSketch(True)
        swModel.ClearSelection2(True)
        swSketchSegment = swSketchMgr.CreateLine(-0.055275, 0.03236, 0.0#, 0.027405, 0.03236, 0.0#)
        swSketchSegment = swSketchMgr.CreateLine(0.027405, 0.03236, 0.0#, 0.027405, -0.026476, 0.0#)
        swSketchSegment = swSketchMgr.CreateLine(0.027405, -0.026476, 0.0#, -0.055275, -0.026476, 0.0#)
        swSketchSegment = swSketchMgr.CreateLine(-0.055275, -0.026476, 0.0#, -0.055275, -0.070758, 0.0#)
        swSketchSegment = swSketchMgr.CreateLine(-0.055275, -0.070758, 0.0#, 0.027405, -0.070758, 0.0#)
        swSketchSegment = swSketchMgr.CreateLine(0.027405, -0.070758, 0.0#, 0.076642, 0.03236, 0.0#)
        swModel.ClearSelection2(True)
```

```
        Stop
        ' Examine the sketch before trimming
        ' the selected lines to a corner
        ' Press F5
        ' Select two lines to trim to a corner
```

```
        status = swModelDocExt.SelectByID("Line6", "SKETCHSEGMENT", 0.0391723509933775, -0.0466042594822396, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("Line3", "SKETCHSEGMENT", 0.0193539283564118, -0.0264761739915713, 0, True, 0, Nothing, 0)
        status = swSketchMgr.SketchTrim(swSketchTrimChoice_e.swSketchTrimCorner, 0, 0, 0)
        swModel.ClearSelection2(True)
```

```
    End Sub
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

## Turn Cameras On and Off Example (C#)

This example shows how to turn cameras on and off.

```
//-----
// Preconditions: Open a SOLIDWORKS Design model that contains at least one camera.
//
// Postconditions: The cameras are turned on and off.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace CameraType_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        // Utility struct to combine the name and ID of a camera
        public struct Camera_t
        {
            public string Name;
            public int Id;
        }

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelView swModelView = default(ModelView);
            Camera swCamera = default(Camera);
            int lNumCameras = 0;
            int lCameraId = 0;
            Feature swFeature = default(Feature);
            Camera_t[] aCameras = null;
            bool bValue = false;
            string sFeatureName = null;
            int lTypeOfCamera = 0;
            int lTypeOfCameraPosition = 0;

            // Get active document
            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Get the active model view for the active document
            swModelView = (ModelView)swModel.ActiveView;

            // Check if a camera view is active
            swCamera = swModelView.Camera;

            Debug.Print("");

            if ((swCamera == null))
            {
                Debug.Print("No active camera.");
            }
            else
            {
                Debug.Print("Camera is active.");
                Debug.Print("  Name = " + swCamera.ID);
            }

            // Turn off camera view
            swModelView.Camera = null;

            // Get the number of cameras
            lNumCameras = swModel.Extension.GetCameraCount();

            Debug.Print("");
            Debug.Print("Number of cameras = " + lNumCameras);

            aCameras = new Camera_t[lNumCameras];

            for (lCameraId = 0; lCameraId <= (lNumCameras - 1); lCameraId++)
            {
                // Valid ID:
                // 0 <= ID <= (ModelDocExtension::GetCameraCount - 1)
                // IDs are reassigned if a camera is deleted
                aCameras[lCameraId].Id = lCameraId;

                // Get the camera
                swCamera = swModel.Extension.GetCameraById(lCameraId);

                // A camera is a feature
            }
        }
    }
}
```

```

swFeature = (Feature)swCamera;

// Get the names of the feature and camera
sFeatureName = swFeature.Name;

aCameras[lCameraId].Name = sFeatureName;

Debug.Print("");
Debug.Print("Type of feature = " + swFeature.GetTypeName());
Debug.Print("Camera name = " + sFeatureName);

// Get the type of camera
lTypeOfCamera = swCamera.Type;

if ((lTypeOfCamera == 1))
{
    Debug.Print("Type of camera = Aimed at target");
}
else
{
    Debug.Print("Type of camera = Floating");
}

// Get the type of camera position
lTypeOfCameraPosition = swCamera.PositionType;

if ((lTypeOfCameraPosition == 1))
{
    Debug.Print("Type of camera position = Cartesian");
}
else
{
    Debug.Print("Type of camera position = Spherical");
}
}

// Now switch the view to each camera
for (lCameraId = 0; lCameraId <= (lNumCameras - 1); lCameraId++)
{
    bValue = swModelView.SetCameraByName(aCameras[lCameraId].Name);

    if ((bValue == false))
    {
        Debug.Print("Failed to set model view to use camera " + aCameras[lCameraId].Name);
    }
    else
    {
        Debug.Print("Model view set to use camera " + aCameras[lCameraId].Name);
    }

    swModelView.Camera = null;
}
}

public SldWorks swApp;
}
}

```

## Turn Cameras On and Off Example (VBA)

This example shows how to turn cameras on and off.

```
'-----
' Preconditions: Open a SOLIDWORKS Design model that contains at least one camera.
'
' Postconditions: The cameras are turned on and off.
'-----
```

## Option Explicit

```
' Utility struct to combine the name and ID of a camera
```

```
Type Camera_t
    Name As String
    Id As Long
End Type
```

```
Dim swApp As SldWorks.SldWorks
```

```
Sub main()
```

```
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelView As SldWorks.ModelView
    Dim swCamera As SldWorks.Camera
    Dim lNumCameras As Long
    Dim lCameraId As Long
    Dim swFeature As SldWorks.Feature
    Dim aCameras() As Camera_t
    Dim bValue As Boolean
    Dim sFeatureName As String
    Dim lTypeOfCamera As Long
    Dim lTypeOfCameraPosition As Long
```

```
    ' Connect to SOLIDWORKS
    Set swApp = Application.SldWorks
```

```
    ' Get active document
    Set swModel = swApp.ActiveDoc
```

```
    ' Get the active model view for the active document
    Set swModelView = swModel.ActiveView
```

```
    ' Check if a camera view is active
    Set swCamera = swModelView.Camera
```

```
    Debug.Print
```

```
    If (swCamera Is Nothing) Then
        Debug.Print "No active camera."
```

```
    Else
        Debug.Print "Camera is active."
        Debug.Print "    Name = " & swCamera.Id
    End If
```

```
    ' Turn off camera view
    swModelView.Camera = Nothing
```

```
    ' Get the number of cameras
    lNumCameras = swModel.Extension.GetCameraCount
```

```
    Debug.Print
    Debug.Print "Number of cameras = " & lNumCameras
```

```
    ReDim aCameras(lNumCameras - 1)
```

```
    For lCameraId = 0 To (lNumCameras - 1)
```

```
        ' Valid ID:
        ' 0 <= ID <= (ModelDocExtension::GetCameraCount - 1)
        ' IDs are reassigned if a camera is deleted
        aCameras(lCameraId).Id = lCameraId
```

```
        ' Get the camera
        Set swCamera = swModel.Extension.GetCameraById(lCameraId)
```

```
        ' A camera is a feature
        Set swFeature = swCamera
```

```
        ' Get the names of the feature and camera
        sFeatureName = swFeature.Name
```

```
        aCameras(lCameraId).Name = sFeatureName
```

```
        Debug.Print
        Debug.Print "Type of feature = "; swFeature.GetTypeName
        Debug.Print "Camera name = " & sFeatureName
```

```
        ' Get the type of camera
        lTypeOfCamera = swCamera.Type
```

```
        If (lTypeOfCamera = 1) Then
            Debug.Print "Type of camera = Aimed at target"
```

```
        Else
            Debug.Print "Type of camera = Floating"
        End If
```

```

' Get the type of camera position
lTypeOfCameraPosition = swCamera.PositionType
If (lTypeOfCameraPosition = 1) Then
    Debug.Print "Type of camera position = Cartesian"
Else
    Debug.Print "Type of camera position = Spherical"
End If
Next lCameraId

' Now switch the view to each camera
For lCameraId = 0 To (lNumCameras - 1)
    bValue = swModelView.SetCameraByName(aCameras(lCameraId).Name)
    If (bValue = False) Then
        Debug.Print "Failed to set model view to use camera " & aCameras(lCameraId).Name
    Else
        Debug.Print "Model view set to use camera " & aCameras(lCameraId).Name
    End If
    swModelView.Camera = Nothing
Next lCameraId
End Sub

```

## Turn Cameras On and Off Example (VB.NET)

This example shows how to turn cameras on and off.

```
'-----
' Preconditions: Open a SOLIDWORKS Design model that contains at least one camera.
' Postconditions: The cameras are turned on and off.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

Partial Class SolidWorksMacro

```
    ' Utility struct to combine the name and ID of a camera
    Structure Camera_t
        Dim Name As String
        Dim Id As Integer
    End Structure

    Sub main()

        Dim swModel As ModelDoc2
        Dim swModelView As ModelView
        Dim swCamera As Camera
        Dim lNumCameras As Integer
        Dim lCameraId As Integer
        Dim swFeature As Feature
        Dim aCameras() As Camera_t
        Dim bValue As Boolean
        Dim sFeatureName As String
        Dim lTypeOfCamera As Integer
        Dim lTypeOfCameraPosition As Integer

        ' Get active document
        swModel = swApp.ActiveDoc

        ' Get the active model view for the active document
        swModelView = swModel.ActiveView

        ' Check if a camera view is active
        swCamera = swModelView.Camera

        Debug.Print("")

        If (swCamera Is Nothing) Then
            Debug.Print("No active camera.")
        Else
            Debug.Print("Camera is active.")
            Debug.Print("  Name = " & swCamera.ID)
        End If

        ' Turn off camera view
        swModelView.Camera = Nothing

        ' Get the number of cameras
        lNumCameras = swModel.Extension.GetCameraCount

        Debug.Print("")
        Debug.Print("Number of cameras = " & lNumCameras)

        ReDim aCameras(lNumCameras - 1)

        For lCameraId = 0 To (lNumCameras - 1)

            ' Valid ID:
            ' 0 <= ID <= (ModelDocExtension::GetCameraCount - 1)
            ' IDs are reassigned if a camera is deleted
            aCameras(lCameraId).Id = lCameraId

            ' Get the camera
            swCamera = swModel.Extension.GetCameraById(lCameraId)

            ' A camera is a feature
            swFeature = swCamera

            ' Get the names of the feature and camera
            sFeatureName = swFeature.Name

            aCameras(lCameraId).Name = sFeatureName

            Debug.Print("")
            Debug.Print("Type of feature = " & swFeature.GetTypeName())
            Debug.Print("Camera name = " & sFeatureName)
        End For
    End Sub
End Class
```

```

' Get the type of camera
lTypeOfCamera = swCamera.Type

If (lTypeOfCamera = 1) Then
    Debug.Print("Type of camera = Aimed at target")
Else
    Debug.Print("Type of camera = Floating")
End If

' Get the type of camera position
lTypeOfCameraPosition = swCamera.PositionType

If (lTypeOfCameraPosition = 1) Then
    Debug.Print("Type of camera position = Cartesian")
Else
    Debug.Print("Type of camera position = Spherical")
End If

Next lCameraId

' Now switch the view to each camera
For lCameraId = 0 To (lNumCameras - 1)

    bValue = swModelView.SetCameraByName(aCameras(lCameraId).Name)

    If (bValue = False) Then
        Debug.Print("Failed to set model view to use camera " & aCameras(lCameraId).Name)
    Else
        Debug.Print("Model view set to use camera " & aCameras(lCameraId).Name)
    End If

    swModelView.Camera = Nothing

Next lCameraId

End Sub

Public swApp As SldWorks

End Class

```

## Turn Lights On Example (VBA)

This example shows how to:

- turn all of the lights on.
- pack two integers into a double value.

```

'-----
' Preconditions:
' 1. Open a part or assembly.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Turns on all of the lights.
' 2. Examine the Immediate window.
'-----
Option Explicit

' Define two types
Type DoubleRec
    dValue As Double
End Type

Type Int2Rec
    iLower As Long ' Assuming that a C int has 4 bytes
    iUpper As Long
End Type

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swFeatMgr As SldWorks.FeatureManager
    Dim nLightCount As Long
    Dim vLightProp As Variant
    Dim dr As DoubleRec
    Dim i2r As Int2Rec
    Dim i As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swFeatMgr = swModel.FeatureManager

    ' Forcibly turn lights on
    i2r.iLower = 0
    i2r.iUpper = 0
    LSet dr = i2r

    swFeatMgr.EnableFeatureTree = False

    Debug.Print "File = " & swModel.GetPathName
    nLightCount = swModel.GetLightSourceCount
    For i = 0 To nLightCount - 1
        vLightProp = swModel.LightSourcePropertyValues(i)
        Debug.Print " SOLIDWORKS Design name for light = " & swModel.GetLightSourceName(i)
        Debug.Print " User name for light = " & swModel.LightSourceUserName(i)
        vLightProp(17) = dr.dValue
        swModel.LightSourcePropertyValues(i) = vLightProp
    Next i

    swFeatMgr.EnableFeatureTree = True

    swFeatMgr.UpdateFeatureTree

    swModel.EditRebuild3

End Sub

```

## Undo Deleted Note and Fire Undo Pre- and Post-Notify Events Example (C#)

This example demonstrates firing Undo pre- and post-notification events in a drawing document.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\AutoCAD\7550-021.slddrw.
//
// Postconditions:
// 1. Selects and deletes the note in Drawing View1.
// 2. Undoes the deleted note.
// 3. Fires pre-notification event indicating that an undo action is about to
//    occur and fires post-notification event indicating that an undo
//    action occurred.
// 4. Click OK to close each message box.
//
// NOTE: Because the drawing is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Collections;
using System.Windows.Forms;
namespace UndoPostNotifyDrawingCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public DrawingDoc swDrawing;

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            bool boolstatus = false;
            Hashtable openDrawing = default(Hashtable);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Event notification
            swDrawing = (DrawingDoc)swModel;
            openDrawing = new Hashtable();
            AttachEventHandlers();

            // Activate the drawing view that contains
            // the note you want to delete
            boolstatus = swDrawing.ActivateView("Drawing View3");
            boolstatus = swModelDocExt.SelectByID2("DetailItem77@Drawing View3", "NOTE", 0.3058741216774, 0.1870419466786, 0, false, 0,
            null, 0);

            // Delete the selected note
            swModel.EditDelete();

            // Undo deletion of note
            swModel.EditUndo2(1);

            // Post-notification is fired

            // Rebuild the drawing
            swModel.ForceRebuild3(true);
        }

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }
    }
}
```

```

}

public void AttachSWEvents()
{
    swDrawing.UndoPostNotify += this.swDrawing_UndoPostNotify;
    swDrawing.UndoPreNotify += this.swDrawing_UndoPreNotify;
}

private int swDrawing_UndoPostNotify()
{
    // Display message after Undo
    // NOTE: Because the message box may be displayed
    // behind an opened window, you might not see it.
    // If so, then check the Taskbar.
    MessageBox.Show("An undo post-notification event has been fired.");
    return 1;
}

private int b()
{
    // Display message after Undo
    // NOTE: Because the message box may be displayed
    // behind an opened window, you might not see it.
    // If so, then check the Taskbar.
    MessageBox.Show("An Undo pre-notification event has been fired.");
    return 1;
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

## Undo Deleted Note and Fire Undo Pre- and Post-notify Events Example (VBA)

This example demonstrates firing Undo pre- and post-notifications in a drawing document.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\AutoCAD\7550-021.slddrw.
' 2. Copy and paste Main into the main module.
' 3. Click Insert > Class Module and copy and paste Class1 into that module.
'
' Postconditions:
' 1. Selects and deletes the note in Drawing View1.
' 2. Undoes the deleted note.
' 3. Fires pre-notification event indicating that an undo action is about to
'    occur and fires post-notification event indicating that an undo
'    action occurred.
' 4. Click OK to close each message box.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
'Main
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swDrawing As SldWorks.DrawingDoc
Dim boolstatus As Boolean
Dim swDrawingEvents As Class1

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    ' Event notification
    Set swDrawing = swModel
    Set swDrawingEvents = New Class1
    Set swDrawingEvents.swDrawing = swApp.ActiveDoc

    ' Activate the drawing view that contains
    ' the note you want to delete
    boolstatus = swDrawing.ActivateView("Drawing View1")
    boolstatus = swModelDocExt.SelectByID2("DetailItem66@Drawing View1", "NOTE", 0.357212931256279, 0.178674480463097, 0, False, 0, Nothing, 0)

    ' Delete the selected note
    swModel.EditDelete

    ' Undo deletion of note
    ' Pre- and post-notifications
    ' are fired
    swModel.EditUndo2 1

    ' Rebuild the drawing
    swModel.ForceRebuild3 True

End Sub

'-----
'Class1
Option Explicit

Public WithEvents swDrawing As SldWorks.DrawingDoc

Private Function swDrawing_UndoPostNotify() As Long
'Display message after undo action occurs
MsgBox "An Undo post-notification event has been fired."
End Function

Private Function swDrawing_UndoPreNotify() As Long
'Display message after undo action occurs
MsgBox "An Undo pre-notification event has been fired."
End Function
```

[Go to top](#)

[Go to top](#)